

Введение

В современном мире технологии достигли того уровня, что большой объем информации хранится в киберпространстве и благодаря такому типу хранилища людям во всем мире стало легче получать доступ к информации, чем когда-либо прежде. До конца прошлого века сбор информации был сложной и долгой задачей, однако теперь люди могут получить доступ к огромному объему информации за несколько минут, а с развитием высоких технологий в настоящее время в мире все больше организаций зависят от своих информационных систем и от информационной безопасности этих систем. Информационная безопасность означает защиту информации и информационных систем от несанкционированного доступа, записи, использования, прочтения, уничтожения или модификации. Криптография используется в информационной безопасности для защиты информации от несанкционированного или случайного раскрытия, во время передачи информации или пока информация находится в хранилище. Криптография неразрывно связана с передачей данных, и помимо обеспечения авторизации пользователей, она гарантирует целостность передаваемой информации и конфиденциальность. Очевидно, что для таких организаций как банки, военные ведомства, государственные учреждения применение криптостойких систем необходимо, и криптография используется различными учреждениями ежедневно.

Криптографическая система - система обеспечения безопасности защищаемой системы, которая использует криптографические средства и методы. Существует два основных типа криптографических систем: симметричные и асимметричные. Системы с симметричными ключами требуют, чтобы и отправитель, и получатель имели один и тот же ключ. Этот ключ используется отправителем для шифрования данных и снова получателем для расшифровки данных. У каждого пользователя есть как

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		6

открытый, так и закрытый ключ. Сообщения зашифровываются одним ключом и могут быть расшифрованы только другим ключом. С развитием квантовых компьютеров увеличивается и риск взлома таких доквантовых криптосистем с открытым ключом как RSA (криптосистема названная в честь трех ее изобретателей Ривеста, Шамира и Адлемана), криптосистемы на эллиптических кривых (ЕСС). Поэтому постквантовые системы шифрования становятся актуальными как никогда. Криптографическая система с открытым ключом NTRUEncrypt может сохранить свою актуальность и в постквантовую эпоху за счет того, что данная система основана на решеточной криптосистеме и ее стойкость обеспечивается сложностью поиска кратчайшего вектора решетки. До сих пор не существует алгоритма, который бы решал эту задачу, поэтому криптографическая система NTRUEncrypt может сохранить свою актуальность не только в настоящий момент, но и в будущем.

Объектом исследования являются постквантовые криптографические системы шифрования информации.

Предмет исследования: криптографическая система с открытым ключом NTRUEncrypt.

Целью данной работы является разработка и реализация криптографической системы NTRUEncrypt с открытым ключом.

В соответствии с поставленной целью были определены задачи:

- провести анализ предметной области;
- изучить уже существующие доквантовые криптографические системы;
- разработать программную реализацию алгоритма NTRUEncrypt с интерфейсом пользователя;
- протестировать разработанное программное средство при различных параметрах и сравнить скорость генерации ключа, шифрования и расшифрования данных реализованного программного продукта с другими криптографическими системами.

1 Основные понятия защиты информации

1.1 Актуальность защиты информации

Информационная безопасность - это общий термин для обозначения того, как отдельные лица и организации защищают свои ценные активы, будь то личные данные, интеллектуальная собственность и т.д. Такая информация может храниться разными способами - например, это могут быть физические файлы на серверах и жестких дисках, на личных устройствах или в облаке. Способы защиты могут быть разными; нельзя применить те же механизмы защиты для бумажных документов, как для цифровых файлов. Первые должны храниться в ящике и быть доступными только для определенного персонала, будь то размещение файлов в закрытом помещении или запирание самого ящика. Цифровые файлы требуют технических средств защиты, таких как контроль доступа, чтобы гарантировать, что только ограниченное число пользователей могут их просматривать. Общий принцип остается прежним - реализация элементов управления, ограничение круг лиц, которые могут просматривать данные, однако методы различаются. Информационная безопасность относится к общей практике защиты персональной информации и способам к ее достижению. Информационная безопасность стала очень важна почти во всех организациях. Это связано с тем, что доступ к информации и ее использование стали проще с появлением информационных технологий, таких как Интернет, который используются во многих организациях.

Для того чтобы удостовериться в том, что информационная безопасность хорошо организована, организации необходимо обеспечить надлежащую защиту своей информации и поддерживать высокий уровень информационной безопасности. Информация в организации нуждается в защите, потому что она имеет ценность для организации. Организация обычно

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		8

ведет организационные и индивидуальные записи. Например, организация может хранить конфиденциальную информацию о своих сотрудниках, информацию о заработной плате, финансовую отчетность, а также бизнес-планы. Так же организация может хранить коммерческую тайну, исследования и другую информацию, которая дает конкурентное преимущество их компании. Помимо этого, для отдельных лиц организация хранит конфиденциальную информацию об их личных данных на компьютерах, которые обычно выполняют онлайн-функции, такие как банковское дело, покупки и социальные сети, обменивая конфиденциальную информацию с другими через Интернет. По мере того, как все больше и больше этой информации хранится и обрабатывается в электронном виде и передается по сетям компании или через Интернет, риск несанкционированного доступа будет возрастать. Если информация не защищена, доступ к ней может получить кто угодно. Кроме того, если информация попадет в чужие руки через, например, воровство или кражу личных данных, это может обрушить бизнес и нанести вред всей организации.

1.2 Цели информационной безопасности

Основными компонентами информационной безопасности являются: конфиденциальность, целостность и доступность.

– конфиденциальность – означает, что данными могут обладать только те люди, которые имеют к ним доступ; чтобы обеспечить конфиденциальность, необходимо иметь возможность определять, кто пытается получить доступ к данным, и блокировать попытки тех, кто не авторизован.

– целостность – означает поддержание данных в правильном состоянии и предотвращение их ненадлежащего изменения, случайно или злонамеренно. Многие методы, обеспечивающие конфиденциальность, также защищают

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		9

целостность данных (злоумышленник не может изменить данные, к которым он не имеет доступа), но есть и другие инструменты, которые помогают обеспечить защиту целостности: контрольные суммы могут помочь проверить данные, а также программное обеспечение для контроля версий и частое резервное копирование могут помочь восстановить данные, если это необходимо.

– доступность – означает соответствие сетевых и вычислительных ресурсов ожидаемому объему доступа к данным и внедрение политики резервного копирования в целях аварийного восстановления. Также к данным не могут получить доступ неавторизованные пользователи.

1.3 Угрозы информационной безопасности

Примером источников угроз информационной безопасности являются:

– антропогенный. Происходят в результате случайных или умышленных действий пользователя;

– техногенный. Происходят в результате использования старых программных средств или в результате программных ошибок;

– стихийные. Они возникают в результате стихийных бедствий.

Антропогенные источники можно разделить на следующие категории:

– случайные и преднамеренные действия пользователей.

– на внутренние и внешние источники. Несанкционированные действия третьих лиц выступают в качестве внешних источников, действия сотрудников выступают в качестве внутренних источников;

Сегодня существует довольно много различных способов утечки информации и незаконных способов доступа к ней. Вот некоторые примеры:

– перехват данных при их передаче;

– изменение данных;

– замена авторства сообщения;

- злоумышленник может использовать различные недостатки операционной системы или установленного программного обеспечения;
- несанкционированный доступ к оборудованию и линиям связи;

Для обеспечения необходимого уровня безопасности информационных систем необходимо использовать комплекс организационно-технических мер и программно-аппаратных средств, направленных на предотвращение угроз безопасности.

1.4 Кибербезопасность информационных систем

Кибербезопасность - это особый тип информационной безопасности, который относится к способам, за счет которых организации обеспечивают защиту цифровой информации такой как сети, программы, устройства, серверы и т.д. Несмотря на то, что это лишь один из аспектов информационной безопасности (наряду с физической безопасностью), ему уделяется больше всего внимание, так как киберугрозы гораздо более вероятны, чем физические. Вредоносное программное обеспечение (ПО), взлом и внутренние ошибки являются основными причинами утечки данных, поэтому имеет смысл уделять внимание защите, которая уменьшает эти риски. Однако, это не означает, что кибербезопасность и физическая безопасность не имеют ничего общего.

Если рассматривать угрозу кражи устройств, то необходимы меры физической безопасности, чтобы устройство не попало в чужие руки. В первую очередь это будет осуществляться в форме правил, инструктирующих сотрудников о том, как обращаться со своими устройствами, вне помещения. Тем не менее, они должны быть дополнены мерами кибербезопасности, которые защитят организацию в случае кражи подобного устройства. Такие меры могут включать шифрование конфиденциальной информации, парольную защиту устройства и баз данных.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		11

1.5 Способы и виды защиты информации

Для сохранения информации в секрете используют методы шифрование или, другими словами, применяют криптографию. Для осуществления шифрования данных применяют алгоритм или используют устройство, которое реализует в свою очередь определенный криптографический алгоритм.

Криптография – это наука о различных способах защиты информации, к которой необходимо ограничить доступ, то есть необходимо сохранить конфиденциальность данной информации.

Криптографические методы защиты информации – это изменение или преобразование данных для их дальнейшей транспортировки доверенному лицу. Принцип этих методов заключается в следующем. Исходное сообщение зашифровывается, принимая форму закрытого текста (графического изображения), которое называется шифротекстом. Затем, шифротекст отправляется пользователю по защищенному или даже допустимо по незащищенному каналу передачи данных.

Получив шифротекст, авторизованный пользователь выполняет обратное преобразование или расшифрование принятого сообщения и восстанавливает исходный текст.

Основные криптографические алгоритмы делятся на три типа:

- криптография с закрытым ключом: используется один и тот же ключ для шифрования и расшифрования; данный способ является симметричным шифрованием. Может использоваться для того, чтобы обеспечить конфиденциальности.

- криптография с публичным ключом: используются разные ключи - один предназначен для шифрования, а другой – для расшифрования; данный способ является асимметричным шифрованием. Может использоваться для аутентификации.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		12

– хеш-функции: используют математическое преобразование для необратимого «шифрования» информации, предоставляя цифровой отпечаток пальца. В основном используется для целостности сообщения [1].

Повышенная безопасность данных, обеспечиваемая криптографией с открытым ключом, является ее основным преимуществом. Криптография с открытым ключом остается наиболее безопасным протоколом (по сравнению с криптографией с закрытым ключом), потому что пользователям никогда не нужно передавать или раскрывать свои закрытые ключи кому-либо, что снижает шансы киберпреступников обнаружить секретный ключ человека во время передачи.

Криптография с открытым ключом также обеспечивает цифровые подписи и требует, чтобы каждый пользователь отвечал за защиту своего закрытого ключа, в то время как системы закрытого ключа требуют, чтобы пользователи делились секретными ключами и, возможно, даже доверяли третьим сторонам для передачи [1]. С помощью системы секретных ключей отправители могут заявить, что общий секретный ключ был скомпрометирован одной из сторон, участвующих в процессе.

Шифрование с открытым ключом имеет преимущество перед системами шифрования с симметричным закрытым ключом, потому что она обходит логику и риски, присущие тайному обмену ключами. За счет своей масштабируемости и повышенной безопасности по сравнению с криптографией с закрытым ключом, криптография с открытым ключом очень популярна и широко используется сегодня в корпоративных средах.

Любая криптографическая система с открытым ключом зависит от надежной генерации ключа. Создание надежных ключей возможно только в том случае, если есть доступ к высококачественной случайности.

Компьютеры — это детерминированные машины. Предполагается, что при одинаковых сериях инструкций они производят одинаковые результаты. Есть много источников энтропии, которые компьютер может использовать для

генерации случайности. При загрузке операционная система поддерживает пул энтропии, которую она собирает, захватывая случайный шум, такой как показания температуры, движения мыши и данные о времени. Он смешивает все эти данные вместе в пул энтропии. Затем эта энтропия проходит через псевдослучайную функцию (например, хеш-функцию) для создания серии случайных байтов. В системах на основе Unix есть специальный файл, который предоставляет поток этих случайных данных, которые можно использовать для генерации криптографического ключа.

Недостаточная энтропия при генерации ключей привела ко многим атакам на криптосистемы [2]. Одним из таких примеров была ошибка в модуле Android, из-за которой модуль Java выводил случайные числа с низкой энтропией. Это привело к тому, что многие крупные биткойн-приложения генерировали незащищенные закрытые ключи, многие из которых были быстро взломаны. Также были многочисленные сообщения о ключах, сгенерированных с использованием различных специальных эвристик, которые обычно взламываются. Когда дело доходит до криптовалют, неаккуратная генерация ключей приводит к краже и потере средств.

Большинство алгоритмов цифровой подписи требуют, чтобы подписывающая сторона генерировала некоторую случайность, чтобы сам процесс подписания был безопасным. Эта случайность должна привести к тому, что каждая подпись будет отличаться, даже если это одно и то же подписываемое сообщение или один и тот же ключ. Если подписывающая сторона не генерирует случайное число с высокой энтропией во время подписания, становится возможным взломать закрытый ключ после наблюдения достаточного количества подписей.

Фактически, было несколько случаев, когда случайные числа повторно использовались в нескольких подписях. Если это когда-нибудь случится, то вычисление закрытого ключа с помощью алгебры старших классов станет тривиальной задачей.

1.6 Квантовая криптография

В современном компьютерном мире возможность шифровать сообщения не только необходима для того, чтобы хранить некоторую информацию в секрете от других, но и является ключевой частью таких технологий, как криптовалюты, такие как Биткойн. Более того, скандалы вокруг повсеместного прослушивания спецслужбами показали миру, что на самом деле ни одна из используемых сегодня процедур не является действительно безопасной. Текущее шифрование основано на алгоритме RSA, который представляет собой «асимметричную» систему шифрования и использует комбинацию открытого ключа и закрытого ключа [3]. Он основан на теории чисел, а алгоритм основан на понятии простых чисел. Когда шифруется сообщение с помощью компьютера, нужно умножить числа, полученные из закрытого ключа и открытого ключа получателя. Получатель может расшифровать большое число, которое отправили, используя свой закрытый и открытый ключ. «Асимметрия» возникает из-за сложности попытки расшифровать сообщение без знания секретных ключей: получить большое число путем подслушивания процесса легко, но тогда разложить большое число на два простых числа совершенно невозможно, если ключи достаточно длинные [3]. Возникает это потому, что факторизация очень большого числа на классическом компьютере займет очень много времени. Хотя строго не доказано, что факторизация является «сложной проблемой» в математическом смысле, т.е. что не существует алгоритма, который решает ее за полиномиальное компьютерное время, таких алгоритмов еще не найдено, и считается, что они не существуют. Фактически, все известные классические алгоритмы занимают экспоненциально большое количество компьютерного времени. Классическая теория информации в сочетании с квантовой физикой образуют такое понятие, как теория квантовой информации. Наибольший интерес в условиях растущей популяризации информационных технологий

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

представляют разделы, которые относятся к квантовым вычислениям и соответственно к квантовой криптографии. На сегодняшний день специалисты значительно продвинулись в данной области и получили довольно серьезные теоретические результаты. Полученные знания и решения нашли свое практическое применение в разделах квантовой криптографии.

1.7 Защита квантовой информации

Квантовую информацию невозможно скопировать [5]. Этот факт, получивший название «теорема о запрете клонирования», был обнаружен только через 70 лет после появления квантовой механики несмотря на то, что он был доказан буквально двумя линиями вычислений. И благодаря этому квантовая криптография физически защищена от перехвата, за исключением технических недостатков из-за шума, ее невозможно взломать [6]. Квантовая криптография уже используется сегодня. На основе протокола, названного в честь создателей Чарльза Беннета и Жюль Брассара (BB84), машины, поставленные швейцарским стартапом Idquantique, сочетают криптографию RSA со случайным ключом, созданным путем квантового распределения ключей.

1.8 Криптография на решетках

Криптография на основе решеток — это общий термин для конструкций криптографических примитивов, которые используют решетки либо в самой конструкции, либо в доказательстве безопасности. Конструкции на основе решеток в настоящее время являются важными кандидатами для постквантовой криптографии [6]. В отличие от более широко используемых и известных схем с открытым ключом, таких как RSA, криптосистемы Диффи-Хеллмана или криптосистемы с эллиптической кривой, которые теоретически

могут быть легко атакованы квантовым компьютером, некоторые конструкции на основе решеток более устойчивы к атакам со стороны классического и квантового компьютера.

1.8.1 Базис решетки

Будем обозначать скалярное произведение как (x, y) . Пусть $B = \{b_1, \dots, b_m\}$ - набор линейно независимых векторов в R^n .

Тогда решетка размерности m – набор линейных комбинаций b_i с целыми коэффициентами [7]:

$$L = Zb_1 + Zb_2 + \dots + Zb_m \quad (1)$$

Базис решетки – это множество линейно независимых векторов, которое ее порождает. Пример решеток представлен на рисунках 1, 2:

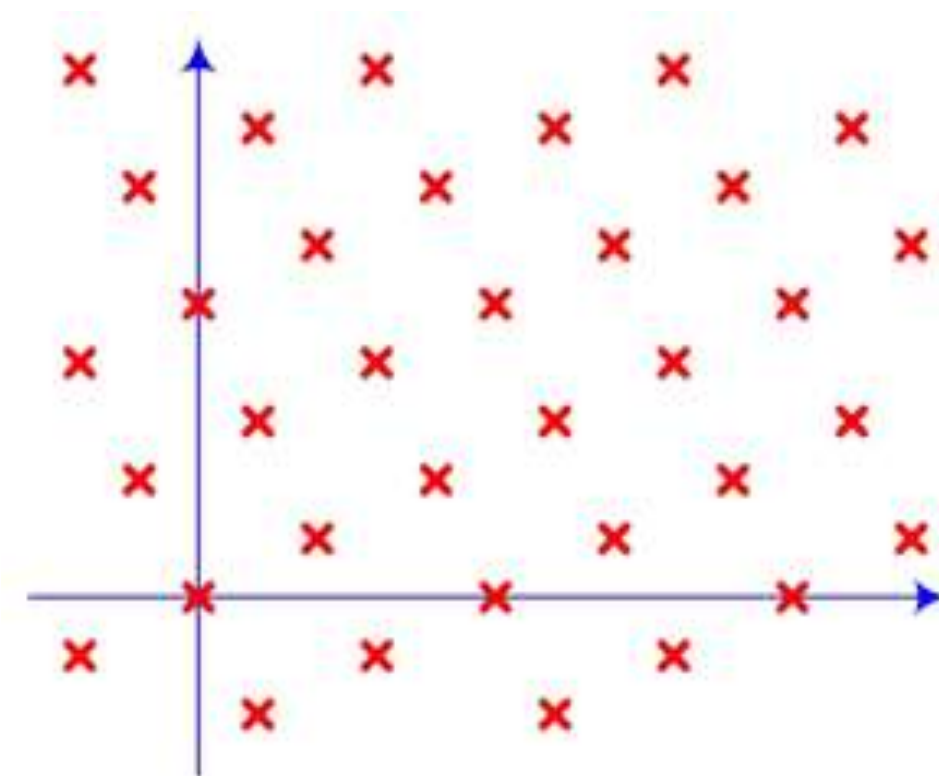


Рисунок 1 — Пример решетки без базиса

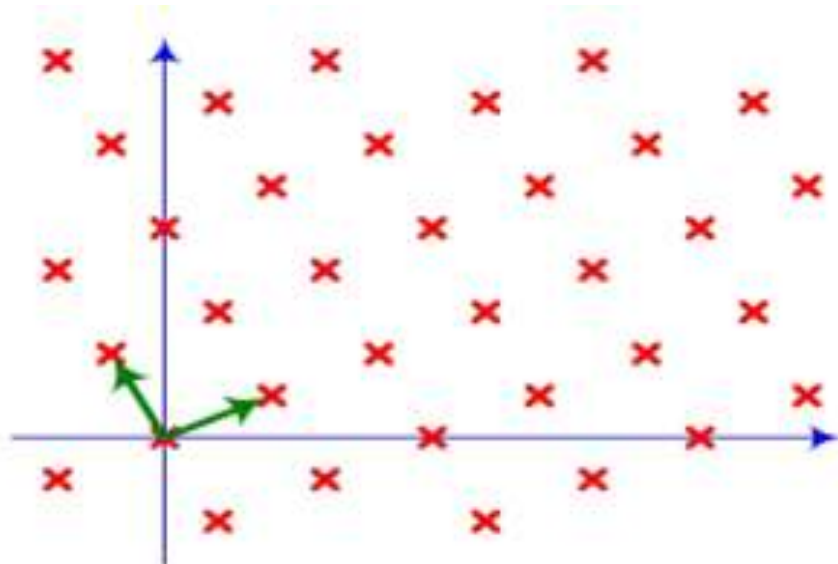


Рисунок 2 — Пример решетки с базисом

Обозначим кратчайший вектор через $\lambda_1(L)$. Многомерное обобщение: $\lambda_1(L)$ – минимальное r , для которого размерность решетки внутри шара радиуса r больше либо равно k . Пример решетки с кратчайшим вектором показан на рисунке 3.

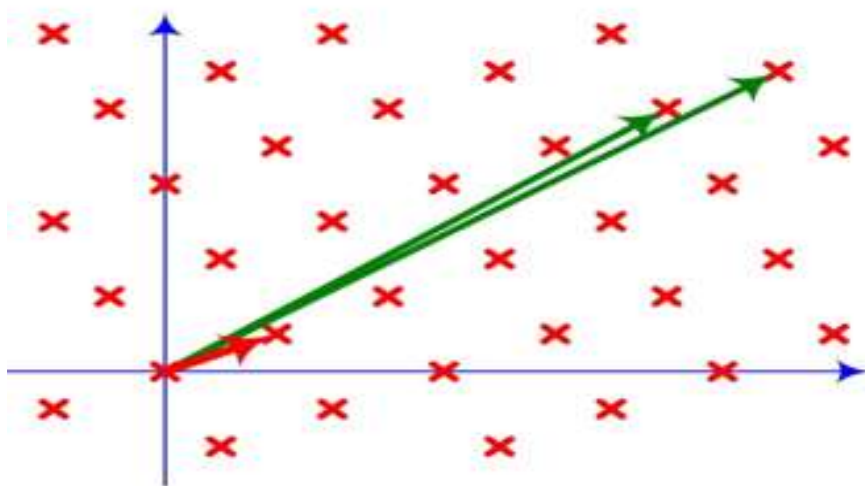


Рисунок 3 — Кратчайший вектор решетки

1.8.2 Кратчайший вектор решетки

Для решения задачи кратчайшего вектора (SVP) при евклидовой норме известно несколько различных подходов, которые можно разделить

на два класса: алгоритмы, требующие сверхэкспоненциального времени ($2^{w(n)}$) и $poly(n)$ память и алгоритмы, требующие экспоненциального времени и пространства ($2^{\theta(n)}$) в размерности решетки [7]. Первый класс алгоритмов, в первую очередь, включает решетчатую нумерацию и сокращение случайной выборки, в то время как второй включает решетчатое просеивание и вычисление ячейки решетки дискретной и гауссовой выборкой. Открытая проблема заключается в том, существуют ли алгоритмы для решения точных SVP, работающие за одно экспоненциальное время ($2^{O(n)}$). Для решения варианта γ – приближения SPV_γ для $\gamma > 1$ для евклидовой нормы наиболее известные подходы основаны на использовании редукции базиса решетки [8]. Для больших $2^{\Omega(n)}$, алгоритм Ленстры – Ленстры – Ловаса (LLL) может найти решение за время, полиномиальное в размерности решетки. Для меньших значений γ , обычно используется блочный алгоритм Коркина-Золотарева (БКЗ), где входные данные алгоритма (размер блока β) определяют временную сложность и качество вывода: для больших коэффициентов аппроксимации γ , небольшого размера блока β достаточно, и алгоритм быстро завершается.

Вывод

В данной главе были рассмотрены и кратко описаны цели и угрозы информационной безопасности, а также виды криптографических алгоритмов. Приведены базовые понятия информационной безопасности, а также кратко дано описание квантового шифрования и криптографии на решетках с определением базиса решетки. Также приведены основные отличия информационной безопасности от кибербезопасности информационных систем. Была проведена оценка защищенности квантовой информации, а именно невозможность ее копирования, объясняющая за счет чего квантовая криптография физически защищена от перехвата.

2 Основные понятия криптосистем

2.1 Криптосистема RSA

Криптосистема RSA — это криптографический алгоритм с открытым ключом, который основан на сложности решения проблемы факторизации больших целых чисел.

Алгоритм создания публичных и закрытых ключей, заключается в следующем следующем:

1. Пользователь случайным образом выбирает два больших простых числа p и q .
2. Затем вычисляется число $n = p * q$.
3. Вычисляет функцию Эйлера от n :

$$\varphi(n) = (p - 1)(q - 1) \quad (2)$$

4. Выбирает число e ($1 < e < \varphi(n)$). Это число подчиняется условию взаимной простоты со значением функции Эйлера. В качестве e выбирается простое число, содержащее относительно небольшое количество единиц в двоичном представлении.

5. На следующем шаге пользователь вычисляет число d , так что:

$$d * e \equiv 1 \pmod{\varphi(n)} \quad (3)$$

6. Пара (e, n) будет открытым ключом.
7. Секретный ключ - (d, n) необходимо хранить в секрете. Процесс шифрования:

1. Есть текст $m \in Z_n$.
2. Шифрование производится по формуле:

$$c = m^e \pmod{n} \quad (4)$$

Расшифровка, то есть получение исходного сообщения m из зашифрованного текста, осуществляется с помощью закрытого ключа по формуле:

$$m = c^d \pmod n \quad (5)$$

2.2 Криптография на основе эллиптических кривых

Криптография на основе эллиптических кривых (ЕСС) в наши дни используется все чаще. ЕСС — это криптосистема, основанная на выполнении умножений по эллиптическим кривым в конечных полях. Ее безопасность основывается на сложности вычисления дискретных логарифмов. Схема подписи, используемая с ЕСС, известна как алгоритм цифровой подписи с эллиптической кривой (ECDSA). Это схема подписи, используемая почти во всех криптовалютах. Почти вся криптография с открытым ключом сегодня стандартизирована на эллиптических кривых, потому что ЕСС намного эффективнее RSA при том же уровне безопасности.

Эллиптические кривые - математически сложные объекты. Различные кривые обладают разными характеристиками защиты, и известно, что у некоторых кривых есть недостатки, поэтому их никогда не следует использовать на практике. Биткойн использует кривую под названием secp256k1, которая сейчас популярна в основном из-за ее принятия Биткойном.

ЕСС, альтернатива RSA, представляет собой мощный криптографический подход. Он обеспечивает безопасность между парами ключей для шифрования с открытым ключом, используя математику эллиптических кривых. Популярность ЕСС постепенно растет из-за меньшего размера ключа и способности поддерживать безопасность. Эта тенденция сохранится, поскольку спрос на устройства, обеспечивающие безопасность,

растет из-за роста размера ключей и использования ограниченных мобильных ресурсов. Вот почему так важно понимать криптографию эллиптических кривых в контексте. В отличие от RSA, ECC основывает свой подход к криптографическим системам с открытым ключом на том, как эллиптические кривые структурируются алгебраически над конечными полями. Таким образом, ECC создает ключи, которые сложнее с математической точки зрения взломать. По этой причине ECC считается реализацией следующего поколения криптографии с открытым ключом и более безопасен, чем RSA. Также имеет смысл использовать ECC для поддержания высокого уровня производительности и безопасности. Это связано с тем, что ECC все более широко используется, поскольку веб-сайты стремятся одновременно повысить онлайн - безопасность данных клиентов и большую мобильную оптимизацию. Больше сайтов, использующих ECC для защиты данных, означает большую потребность в таком кратком руководстве по криптографии с эллиптической кривой. Между размерами ключей ECC и ключей RSA нет линейной зависимости. То есть, размер ключа RSA, который вдвое больше, не переводится в размер ключа ECC, который увеличивается вдвое. Это убедительное различие показывает, что генерация и подпись ключей ECC значительно быстрее, чем для RSA, а также что ECC использует меньше памяти, чем RSA. Кроме того, в отличие от RSA, где оба являются целыми числами, в ECC частный и открытый ключи не подлежат обмену в равной степени. Вместо этого в ECC открытый ключ представляет собой точку на кривой, а закрытый ключ по-прежнему является целым числом [8]. В ECC более быстрое создание ключей и подписей, его скорость расшифрования и шифрования умеренно высокая. ECC обеспечивает меньшую задержку, чем обратная, за счет вычисления подписей в два этапа. ECC предлагает надежные протоколы для обмена аутентифицированными ключами, и эта технология сильна. Главный недостаток ECC заключается в том, что его нелегко безопасно реализовать. По сравнению с RSA, который намного проще как с

					<i>10.05.01.410000.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		22

точки зрения проверки, так и со стороны шифрования, ECC - немного медленнее для накопления результатов. Генерация ключей с RSA происходит медленно, как и расшифровка и подпись, которые не всегда легко реализовать безопасно.

Криптография с открытым ключом работает с использованием алгоритмов, которые легко обрабатывать в одном направлении и трудно обрабатывать в обратном направлении. Например, RSA полагается на тот факт, что умножение простых чисел для получения большего числа легко, в то время как преобразование огромных чисел обратно в исходные простые числа намного сложнее. Однако для обеспечения безопасности RSA необходимы ключи длиной 2048 бит или более. Это замедляет процесс, а также означает, что важен размер ключа [9]. Размер - серьезное преимущество криптографии с эллиптическими кривыми, поскольку он обеспечивает большую мощность для небольших мобильных устройств. Это намного проще и требует меньше энергии для разложения, чем для решения дискретного логарифма эллиптической кривой, поэтому для двух ключей одинакового размера шифрование факторинга RSA более уязвимо. С ECC, можно достичь того же уровня безопасности, используя меньшие ключи. ECC предлагает высокую безопасность с более быстрыми и короткими ключами по сравнению с RSA.

2.3 Квантовые компьютеры

Квантовые компьютеры — это машины, которые используют свойства квантовой физики для хранения данных и выполнения вычислений. Это может быть чрезвычайно полезно для определенных задач, где они могут значительно превзойти даже суперкомпьютеры. Классические компьютеры, в том числе смартфоны и ноутбуки, кодируют информацию в двоичных «битах», которые могут иметь значение 0 или 1. В квантовом компьютере основной единицей памяти является кубит.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		23

Кубиты созданы с использованием физических систем, таких как спин электрона или ориентация фотона [9]. Эти системы могут быть одновременно расположены во многих различных конфигурациях, это свойство известно, как квантовая суперпозиция. Кубиты также могут быть неразрывно связаны вместе с помощью явления, называемого квантовой запутанностью. В результате серия кубитов может одновременно представлять разные вещи.

Например, восьми битов достаточно для классического компьютера для представления любого числа от 0 до 255. Но восьми кубитов достаточно, чтобы квантовый компьютер представлял каждое число от 0 до 255 одновременно [9]. Несколько сотен запутанных кубитов будет достаточно, чтобы представить больше чисел, чем атомов во Вселенной. В ситуациях, когда существует большое количество возможных комбинаций, квантовые компьютеры могут рассматривать их одновременно [10]. Примеры включают попытку найти простые множители очень большого числа или лучший маршрут между двумя местами.

Однако также может быть множество ситуаций, когда классические компьютеры все равно будут превосходить квантовые. Компьютеры будущего могут быть комбинацией обоих этих типов. На данный момент квантовые компьютеры очень чувствительны: тепло, электромагнитные поля и столкновения с молекулами воздуха могут привести к потере квантовых свойств кубита. Этот процесс, известный как квантовая декогеренция, вызывает сбой системы, и это происходит тем быстрее, чем больше задействовано частиц.

Создание оборудования для квантового компьютера - сложная задача. Кубиты по своей природе очень хрупкие и могут очень быстро потерять закодированную в них информацию. Основная задача состоит в том, чтобы сохранить кубиты полностью изолированными от окружающей среды, обеспечивая при этом высокоточный контроль и считывание состояния кубита. Чтобы эффективно отделить кубиты от любого источника шума и,

следовательно, поддерживать более длительное время когерентности, эти системы обычно охлаждаются до чрезвычайно низких температур с использованием жидкого гелия, однако это приводит к высоким эксплуатационным расходам. Есть много разных способов реализовать кубиты, например, захваченные ионы, сверхпроводящие кольца и многое другие. У каждой архитектуры есть свои преимущества и недостатки, и пока не ясно, какой материал кубита является наиболее масштабируемым.

Для обеспечения большого числа параллельно выполняющихся процессов модель квантовых вычислений использует несколько относительно простых правил преобразования входной информации. Иными словами, становится возможным за один проход функции вычислить ее значения для всех аргументов. Данная функция применяется к входным данным. На вход подаются данные, которые представляют собой суперпозицию всех возможных значений аргумента, следовательно, функция должна принять и обработать такую последовательность. В квантовых моделях функция представляет из себя эрмитовую матрицу, то есть такую, которая имеет свою эрмитово-сопряженную, при этом произведение этих матриц даст в итоге единичную. Сопряженную матрицу можно получить в два шага, первый шаг – транспонирование исходной матрицы и второй шаг – вместо исходных элементов подставляются их комплексно-сопряженные. Если умножить получившуюся матрицу на вектор квантового регистра, получится квантовый регистр, такой что сумма квадратов коэффициентов при квантовых состояниях будет равна единице.

2.3.1 Квантовый разряд

Кубит (квантовый бит) может хранить один бит - наименьшую возможную единицу цифровой информации - и является фундаментальным строительным блоком будущего квантового компьютера. Кубиты из

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		25

полупроводниковых материалов являются одними из самых многообещающих кандидатов. Модель базельского кубита: два отдельных электрона (красный) захвачены в квантовой точке. Их спиновые состояния (стрелки) образуют информационные блоки (кубиты). Золотые контакты позволяют электронам удерживаться в стабильных электрических полях. Эта структура имеет размер примерно полмикрона и встроена в полупроводниковый материал - в данном случае атомы галлия и мышьяка.

Как и обычные компьютеры, квантовые компьютеры также обрабатывают и хранят информацию в цифровом виде: информация кодируется с помощью системы, состоящей всего из двух цифр (0 и 1). Каждая из этих цифр содержит один «бит» информации, которую можно сохранить с помощью любой технической системы, которую можно включать и выключать. В компьютерах бит хранится в переключаемой цепи, которая либо включена (ON = 1), либо выключена (OFF = 0). Компьютеры разбивают сложную информацию, такую как числа и текст, на биты, которые затем обрабатываются с помощью множества очень быстрых процессов включения либо выключения. Квантовый компьютер будущего также будет разбивать сложную информацию на биты. Однако вместо использования переключаемых схем он обрабатывает и сохраняет эту информацию с помощью квантовой физической системы, которая может принимать состояния ВКЛ и ВЫКЛ. В то время как переключаемая схема находится в состоянии ВКЛ или ВЫКЛ, в квантовых физических системах эти состояния существуют одновременно. Система такого типа, в которой используется суперпозиция двух состояний, получила название квантовый бит (или кубит).

Кубит сам по себе также содержит всего один бит информации - ровно столько, сколько переключаемая схема. Особые свойства, возникающие в результате суперпозиции, вступают в силу только после того, как кубит соединяется с другими кубитами определенным образом, а именно, с использованием характерного явления квантовой физики, известного как

запутанность. Только когда несколько кубитов объединяются в группы (квантовые регистры), они создают очень мощные системы обработки информации. Электронный спин служит основой для изучения и создания кубитов из полупроводниковых материалов в лабораториях ведущих университетов и промышленных компаний по всему миру. Среди ведущих исследовательских групп - твердотельные специалисты под руководством Loss. Их основная идея - использовать спин электрона в качестве кубита. Так как электрон чрезвычайно мал, а магнитное поле, связанное со спином, чрезвычайно слабое, то его трудно измерить. Однако, поскольку вращение может указывать только «вверх» или «вниз», оно имеет ровно два состояния. Более того, спин электрона подчиняется законам квантовой физики. Таким образом, эта система имеет идеальные предпосылки для того, чтобы служить фундаментальным строительным блоком квантового компьютера. Чтобы спин электрона можно было использовать в качестве кубита, должен быть надежный способ не только определить, но и изменить его направление. Ученые надеются достичь этого с помощью концепции, известной как квантовая точка. Проще говоря, квантовая точка представляет собой сферический объем, обычно диаметром в одну десятитысячную миллиметра, расположенный внутри твердого тела [10]. Свободный электрон (то есть тот, который не связан внутри атома) «заперт внутри» сферы. Окружающее твердое тело состоит из слоев из двух полупроводниковых материалов (таких как кремний и германий) и охлаждается до очень низкой температуры - всего одна десятая градуса выше абсолютного нуля, - а свободный электрон удерживается на месте с помощью электрических полей. Физические системы, предназначенные для использования в качестве кубитов, должны обладать рядом особых свойств в дополнение к их квантово-физическим свойствам: например, кубиты должны быть достаточно маленькими, чтобы разместить 100 миллионов из них на кристалле с площадью один квадратный сантиметр; в противном случае будет невозможно построить управляемый компьютер.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		27

Как показывают эти требования, создание функционального кубита — это сложная задача. Ученые добились значительного прогресса с 1990-х годов, благодаря плану, который Лосс опубликовал в 1998 году. В нем был изложен конкретный метод создания кубитов и их использования для создания квантового компьютера.

2.3.2 Алгоритм Шора

Эквивалентным показателем мощности квантовых компьютеров является количество их кубитов (которые эквивалентны транзисторам классических компьютеров). Большие квантовые компьютеры смогут взломать большинство существующих (классических) механизмов шифрования [10]. Питер Шор показал, как можно разложить числа на множители за полиномиальное время на квантовом компьютере. Современные квантовые компьютеры имеют несколько десятков кубитов, однако для правильной реализации алгоритма Шора потребуется в тысячу раз больше. Фактически, алгоритм Шора является одним из самых сложных квантовых алгоритмов с точки зрения количества необходимых квантовых битов и вентилях, и, по оценкам, пройдет несколько десятилетий, прежде чем будет проведена серьезная факторизация. Тем не менее, алгоритм Шора представляет серьезную угрозу для всего, где важна секретность [11]. От спецслужб до секретов компаний - все это может перестать быть безопасным. Однако квантовая криптография способна противостоять данному алгоритму.

2.3.3 Алгоритм Беннета и Brassarda

Один из алгоритмов квантовой криптографии - алгоритм Беннета и Brassarda, изобретенный в 1984 году. Он основан на одной из наиболее

					<i>10.05.01.410000.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		28

важных особенностей квантовой механики: возможности формирования суперпозиции между состояниями, например, между 0 и 1 квантового бита (кубита). Цель протокола состоит в том, чтобы после выполнения Алиса и Боб оба использовали один и тот же ключ, с помощью которого они могут зашифровать сообщения, но не перехватчик Ева [11].

Протокол работает следующим образом: сначала Алиса создает группу кубитов (например, одиночные фотоны) и подготавливает каждый из них в случайном состоянии. После подготовки кубитов она отправляет их Бобу. Он измеряет каждый кубит в случайно выбранном базисе. Это, в частности, означает, что он мог производить измерения в «неправильном» базисе, то есть не в том, в котором Алиса подготовила кубит [11]. Далее Боб объявляет о своих измерениях, а Алиса подтверждает или нет их корректность. Если Боб измеряет верный базис, это означает, что Алиса и Боб теперь могут обмениваться знаниями о результатах измерения. Например, если Алиса подготовила «1» в z-базисе и Боб измеряет в том же базисе, то он получит «1», и они могут использовать эту «1» в качестве цифры своего ключа. Алиса может подготовить кубит «0» в x-базисе, который является равной суперпозицией «0» и «1» в z-базисе. Когда Боб измеряет в z-базисе, он имеет 50% шанс измерить «0» и 50% шанс измерить «1» — это означает, что Алиса и Боб не могут использовать этот кубит в качестве своего ключа, потому что в 50% случаев у них не было бы того же числа. Алиса и Боб могут использовать все числа, измеренные в верном базисе, как свои ключи [12]. Если Ева перехватит передачу и измерит кубиты перед их отправкой Бобу, она сможет изменить результаты измерения Боба. Боб объявляет результаты своих измерений, а именно части кубитов, измеренных в правильном базисе, а Алиса значения, с которыми она их подготовила. Если теперь они обнаруживают, что некоторые значения не совпадают, они могут заподозрить, что Ева пыталась их перехватить, так как она может делать только то же самое, что и Боб: измерять случайным образом. Если теперь Ева измеряла «неправильный»

базис, а Боб измерял «правильный», может случиться так, что Алиса и Боб не придут к согласию относительно результата измерения. То есть, если Алиса подготавливает «0» в z-базисе, а Ева перехватывает этот кубит и случайным образом выбирает для измерения в x-базисе. У нее есть 50% шанс измерить «0» и 50% шанс измерить «1». Единственное, что может вызвать перехват у Алисы и Боба — это то, что Ева измеряет в той же базисе, что и Алиса, и Боб. Вероятность того, что этого $(0,75)^N$ где N - количество кубитов, использованных для создания ключа, т.е имея всего 100 кубитов в ключе, вероятность того, что он будет перехвачен, составляет всего 1 из 1000000000000000.

2.4 Сравнение криптосистем RSA, NTRUEncrypt и ECC

Большая часть наиболее важной информации сегодня передается через глобальный Интернет, например: электронная почта, сообщения чата, видеоконференции, электронная коммерция и данные онлайн-банкинга. Использование криптосистем с открытым ключом - основной способ защиты таких данных. Наиболее широко используется криптосистема RSA, основанная на сложности факторизации больших чисел, схема Диффи-Хеллмана, алгоритм цифровой подписи (DSA), безопасность которой основана на сложности решения задачи дискретного логарифма в поле, семейство алгоритмов, основанных на эллиптических кривых. Но все они имеют определенные недостатки, основными из которых являются либо относительно низкая скорость работы, либо относительно низкая стойкость при сопоставимых размерах ключей и параметров (схема Диффи-Хеллмана и другие алгоритмы, основанные на дискретном логарифме в поле), или оба других одновременно (RSA). Алгоритм NTRU, все операции которого выполняются в кольце усеченных многочленов, был создан для решения этой проблемы. Криптографическая стойкость NTRU основана на

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		30

сложности задачи поиска короткого вектора в заданной решетке [12]. Все криптографические системы, основанные на задачах факторизации целых чисел, дискретного логарифма и дискретного логарифма в группе эллиптических точек кривые потенциально уязвимы для разработки квантового компьютера подходящего размера, поскольку для такого компьютера известны алгоритмы, которые могут решить эти задачи за полиномиальное время, в зависимости от размера входных данных. В настоящее время нет квантовых алгоритмов с полиномиальной сложностью для NTRU. Только криптосистемы, основанные на алгебраических решетках, остаются практически неуязвимыми для квантового криптоанализа. Одним из основных преимуществ алгоритма NTRU также является очень высокая скорость операций шифрования / расшифрования. По данным компании-разработчика NTRU Security Innovation, этот алгоритм до двухсот раз быстрее алгоритмов эллиптических кривых и RSA [12]. На основании имеющихся на данный момент данных можно сделать промежуточные выводы о высоком уровне защищенности NTRU, который не уступает защищенности алгоритмов на основе эллиптических кривых. Однако из-за относительной новизны и низкой распространенности асимметричных алгоритмов этого класса необходимо провести дополнительные исследования на предмет возможных ошибок и критических уязвимостей, которые можно использовать для разработки эффективных атак. В результате анализа производительности NTRU было установлено, что его производительность намного выше, чем у RSA и ECC. NTRU показал лучшие результаты по производительности, так как очень хорошо поддается распараллеливанию, в отличие от RSA, попытки распараллеливания которого практически не дают прироста скорости. Класс асимметричных алгоритмов, основанных на задачах на алгебраических решетках, по совокупности всех показателей значительно превосходит другие алгоритмы с открытым ключом. Следовательно, именно NTRU во

					<i>10.05.01.410000.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		31

многих областях должен заменить RSA и ECC и стать таким же общепринятым стандартом для асимметричной криптографии.

2.5 Преимущества системы NTRUEncrypt

Алгоритм NTRUEncrypt был разработан ещё в середине 90-х годов. В сравнении с криптосистемой RSA, он не получил широкого распространения, так как сначала необходимо было повысить криптостойкость и производительность этого шифроалгоритма. На текущий момент все выявленные недостатки уже исправлены, и на практике считается, что NTRUEncrypt работает намного быстрее, чем алгоритм RSA. При сравнении скоростей NTRU, RSA и ECC с параметрами, соответствующими уровню безопасности $k = 256$ бит, NTRU на 4 порядка быстрее, чем RSA, и на 3 порядка быстрее, чем ECC [12]. На графике (рисунок 4) наглядно показано, насколько криптосистема NTRUEncrypt превосходит, по количествам операций в секунду, большинство существующих доквантовых алгоритмов.

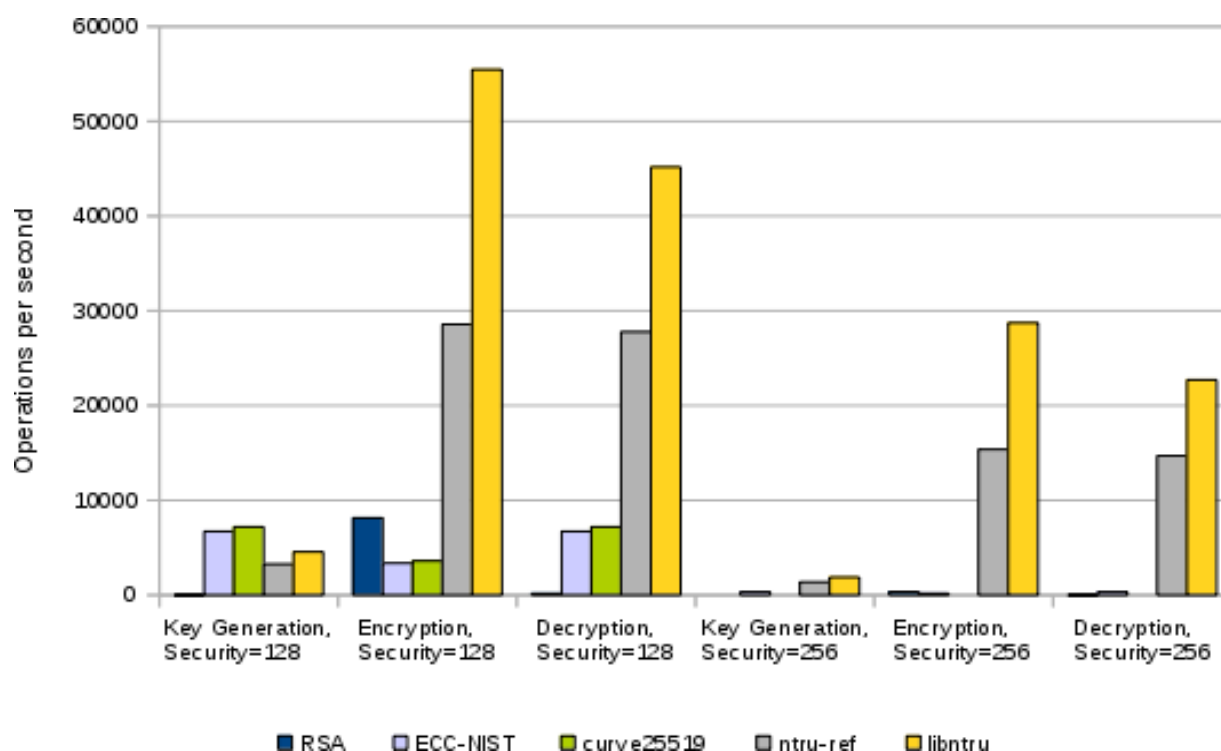


Рисунок 4 – Сравнение быстродействия NTRU с другими алгоритмами

К тому же, криптосистема NTRUEncrypt является постквантовой, и имеет выше в разы криптостойкость к различным атакам. И, как уже было сказано выше, за счет отсутствия алгоритма поиска кратчайшего вектора решётки обеспечивается криптостойкость алгоритма.

Вывод

Представлена идея квантового компьютера, за счет которого может быть осуществлен взлом доквантовых систем шифрования, приведены понятия теории квантовых вычислений, а также кратко дано описание квантового разряда. Был описан алгоритм Шора, Беннета и Brassarda, перечислены основные преимущества криптографической системы NTRUEncrypt по сравнению с доквантовыми криптографическими системами такими как RSA и ECC.

3 Разработка и реализация криптографической системы NTRUEncrypt с открытым ключом

3.1 Общие сведения

NTRUEncrypt, разработанный Гофштейном, Пьефером и Сильверманом, был впервые представлен публике на дополнительной сессии Crypto '96. На данный момент система шифрования NTRUEncrypt является альтернативой схемам шифрования, основанным на факторизации и дискретном логарифмировании по конечным полям и эллиптическим кривым, что отмечено включением его в стандарт P1363. NTRUEncrypt считается жизнеспособным решением постквантового шифрования с открытым ключом. В то время как количество атак и практических усовершенствований NTRUEncrypt росло, теоретическая часть криптографии с доказанной безопасностью, основанная на использовании решёток, остановилась в разработке. Точкой отсчета является 1996 год и признанная трансформация наихудшего сценария в нормальный, что породило устойчивые хеш-функции, взлом которых так же сложен, как взлом некоторых задач над решетками наихудших сценариев. В последние пару лет было представлено большое количество криптографических схем, которые доказали безопасность уровня обучения с ошибками (LWE). Этот список включает защищенные схемы шифрования, основанные на аутентификации, цифровых сигнатурах и т.д. Основным препятствием для криптографии на основе LWE и краткого целочисленного решения (SIS) является ее ограниченная эффективность. Ключ в большинстве случаев содержит случайную матрицу, размеры параметра безопасности линейны; следовательно, требования к пространству и времени являются по крайней мере второй степенью по отношению к параметру безопасности. В 2002 году Миччианчио удалось преобразовать SIS в структурированные матрицы, сохранив при этом преобразование

					<i>10.05.01.410000.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		34

наихудшего случая в нормальное. Наихудший сценарий — это сведение общих решеточных задач к специальному семейству циклических решеток. Работы Миччианчио привели к формированию семейства хеш-функций, устойчивых к атакам на прообраз. Доказано, что полученная хеш-функция устойчива к коллизиям при заданной устойчивости модифицированной задачи нормального сценария; он был назван Ideal-SIS. Он обеспечивает такую же надежность, как и сокращение наихудшего сценария для типичных решеток до специального класса решеток (называемых идеальными). В 2009 году Степе предложил структурированную версию LWE, которая оказалась столь же надежной, как и Ideal-SIS [13].

Параллельно Любашевский в своей независимой работе предложил вариант LWE на кольцах, названный R-LWE, большая гибкость которого позволяет создавать более естественные и эффективные криптографические конструкции. Большинство современных криптографических алгоритмов и протоколов основываются на вычислительной сложности определенных математических задач, таких как факторизация произведений двух больших простых чисел (RSA) или дискретный логарифм по определенным группам (обмен ключами Диффи-Хеллмана, система шифрования Эль-Гамала). Считается, что эти проблемы не имеют эффективных (полиномиальное время) решений, поэтому любой криптографический протокол, основанный на них, должен быть как минимум таким же трудным для взлома. Поскольку можно предположить, что любой потенциальный противник имеет ограниченную вычислительную мощность, можно ожидать, что эти протоколы будут безопасными. Однако это не всегда так. Если противник обладает вычислительной мощностью квантового компьютера, такие алгоритмические проблемы оказываются легко разрешимыми. RSA и многие другие популярные протоколы, такие как обмен ключами Диффи-Хеллмана, схема шифрования Эль-Гамала, DES (стандарт шифрования данных) или ECC, будут нарушены. Их нужно будет заменить на что-то более безопасное,

предполагающее большую вычислительную мощность потенциального противника. Есть несколько алгоритмических проблем, которые сложно решить, как на классических, так и на квантовых компьютерах. Существует множество различных групп протоколов, таких как криптография на основе хеша или кода, криптография с многомерными квадратными уравнениями или криптография на основе решеток [13].

NTRUEncrypt обычно описывается как криптосистема на основе полиномов, включающая продукты свертки. Ее естественно можно рассматривать и как решеточную криптосистему для определенного ограниченного класса решеток. Криптосистема имеет несколько естественных параметров, и, как и во всех практических криптосистемах, можно оптимизировать эти параметры для повышения эффективности, в то же время избегая всех известных криптоаналитических атак. Функциональность криптосистемы не пострадает, если «сбои расшифрования» происходят с очень небольшой вероятностью в случайных сообщениях.

3.2 Генерация открытого ключа

Для примера рассмотрим передачу сообщения виртуальных пользователей: Алисы и Боба. Боб хочет передать сообщение Алисе, для этого ему необходимы открытый и закрытый ключи. Поэтому он случайным образом выбирает два «маленьких» полинома f и g из кольца усеченных многочленов R . «Малость» полиномов означает, что относительно произвольного полинома по модулю q , в котором коэффициенты равномерно распределены, у малого полинома они будут много меньше q . Для определения малости полиномов используются числа df и dg , которые Боб выбирает самостоятельно:

- полином f будет иметь df коэффициентов равных единице, $(df-1)$ коэффициентов равных минус единице и остальные равные нулю;

– полином g будет иметь dg коэффициентов равных единице, столько же коэффициентов равных минус единице и остальные равные нулю.

Боб должен хранить выбранные полиномы в тайне, так как любой, кому они станут известны, сможет расшифровать сообщение. Следующим шагом Боб вычисляет обратные полиномы f_p и f_q по модулю p и q соответственно, такие, что:

$$f * f_p = 1 \pmod{p} \quad (6)$$

$$f * f_q = 1 \pmod{q} \quad (7)$$

В случае, если обратных полиномов не будет существовать, то Бобу нужно вернуться на шаг назад и еще раз выбирать полином f . Пара (f, f_p) является секретным ключом, а открытый ключ h Боб рассчитывает согласно формуле:

$$h = p * f_p * g \pmod{q} \quad (8)$$

3.3 Шифрование данных

Алиса теперь знает открытый ключ h и может отправить любую зашифрованную информацию Бобу. Далее Алисе необходимо представить текст сообщения в виде полинома m . Затем Алиса выбирает некоторый «маленький» полином r , известный как «ослепляющий» полином и вычисляет шифротекст согласно выражению:

$$e = r * h + m \pmod{q} \quad (9)$$

Результатом шифрования станет сумма произведения «ослепляющего» полинома на открытый ключ и исходного полинома m .

3.4 Расшифрование зашифрованных данных

После того как Боб получил от Алисы шифротекст, он может его расшифровать, используя секретный ключ и формулу:

$$a = f * e \pmod{q} \quad (10)$$

Так как Боб вычисляет значение a по модулю числа q , он должен выбрать его коэффициенты из диапазона $(-q/2, q/2]$ и затем вычислить:

$$b = a \pmod{p} \quad (11)$$

Наконец, Боб используя вторую часть секретного ключа получает исходное сообщение от Алисы:

$$c = f_p * b \pmod{p} \quad (12)$$

3.5 Рекомендуемые параметры NTRUEncrypt

Для достижения нужного уровня защиты рекомендуется использовать следующие параметры системы NTRUEncrypt, которые представлены на рисунке 5:

N	q	p	df	dg	dr	Гарантированная стойкость
167	128	3	61	20	18	Умеренный уровень стойкости
251	128	3	50	24	16	Стандартный уровень стойкости
503	256	3	216	72	55	Высочайший уровень стойкости

Рисунок 5 – Рекомендуемые параметры NTRUEncrypt

Сравнение производительностей криптографических систем RSA, ECC, NTRUEncrypt представлено на рисунке 6:

	RSA-1024	ECC-168	NTRU-263
Длина открытого ключа, битов	1024	169	1841
Скорость генерации ключа, мс	1432	65	19,8
Скорость шифрования, мс	4,28	140	1,9
Скорость расшифрования, мс	48,5	67	3,5

Рисунок 6 – Сравнение производительностей NTRUEncrypt, RSA, ECC

Как утверждают разработчики NTRUEncrypt, данная система шифрования на 4 порядка быстрее RSA и на 3 порядка — ECC и может быть использована в «постквантовую» эпоху.

3.6 Сравнение с другими реализациями NTRUEncrypt

Для сравнения реализованной системы шифрования NTRUEncrypt была выбрана реализация криптографической системы NTUEncrypt А.В Селёвой ВКБ61 под руководством профессора, д.ф.-м.н. Л.В Черкесовой, а также онлайн-сервис NTRUEncrypt. Каждая реализация имеет свои достоинства и недостатки, как видно из рисунков 7, 8.

Достоинства	Недостатки
Реализована на языке Python, который имеет все необходимые модули для реализации NTRUEncrypt, а также более доступен для понимания чем другие языки	Пока нет подробного режима программы
Возможность ввода всех необходимых параметров для генерации ключей, шифрования и дешифрования информации	
Ввод и вывод осуществляется непосредственно в самом интерфейсе	

Рисунок 7 — Достоинства и недостатки разработанного программного продукта

Реализация криптографической системы NTUEncrypt А.В Селёвой ВКБ61 под руководством профессора, д.ф.-м.н. Л.В Черкесовой		Онлайн-сервис NTRUEncrypt	
Плюсы	Минусы	Плюсы	Минусы
функциональный интерфейс	нет возможности задавать все параметры для генерации ключей	функциональный интерфейс	необходим доступ в интернет
модифицированная версия программы	нет возможности задавать параметр для полинома g	возможность выбирать размер ключа N	нет возможности задавать все параметры
возможность выбирать размер ключа N и числа q, p			

Рисунок 8 — Сравнения с другими реализациями

3.7 Выбор языка программирования для реализации поставленной в работе задачи

На данном этапе развития информационных технологий существует множество языков программирования, такие как: C#, C++, Kotlin, Python, Pascal и т.д. Так как все операции криптографической системы NTRUEncrypt осуществляются над кольцом усеченных многочленов, был выбран язык программирования Python, который имеет множество подключаемых библиотек, позволяющих проводить расчеты над кольцами и формировать обратные полиномы. Кроме этого, выбранный язык программирования имеет множество встроенных функций, упрощающих процесс написания реализуемого алгоритма шифрования.

Python - это язык программирования с открытым исходным кодом, разработанный в конце 1980-х Гвидо ван Россумом и представленный в 1991 году [13]. Python поддерживается сообществом пользователей и обеспечивает ясный синтаксис, динамический ввод и имеет спектр стандартных библиотек, созданных сторонними разработчиками [13]. Так же, выбранный язык имеет преимущества перед другими языками программирования, такими как:

- по сравнению со многими другими языками, Python имеет простой для чтения синтаксис.
- подходит для разных платформ: как Linux, так и Windows.
- есть реализация интерпретаторов для мобильных устройств и непопулярных систем.
- используется для разработки удобных для автоматизации веб-приложений, игр, математических расчетов, машинного обучения, в области Интернета вещей.

3.8 Описание алгоритма работы программы

В начале работы программы пользователь происходит формирование закрытого и открытого ключа шифрования, которые будут использоваться в дальнейшем при шифровании и расшифрования информации. Данный процесс изображен на рисунке 9.

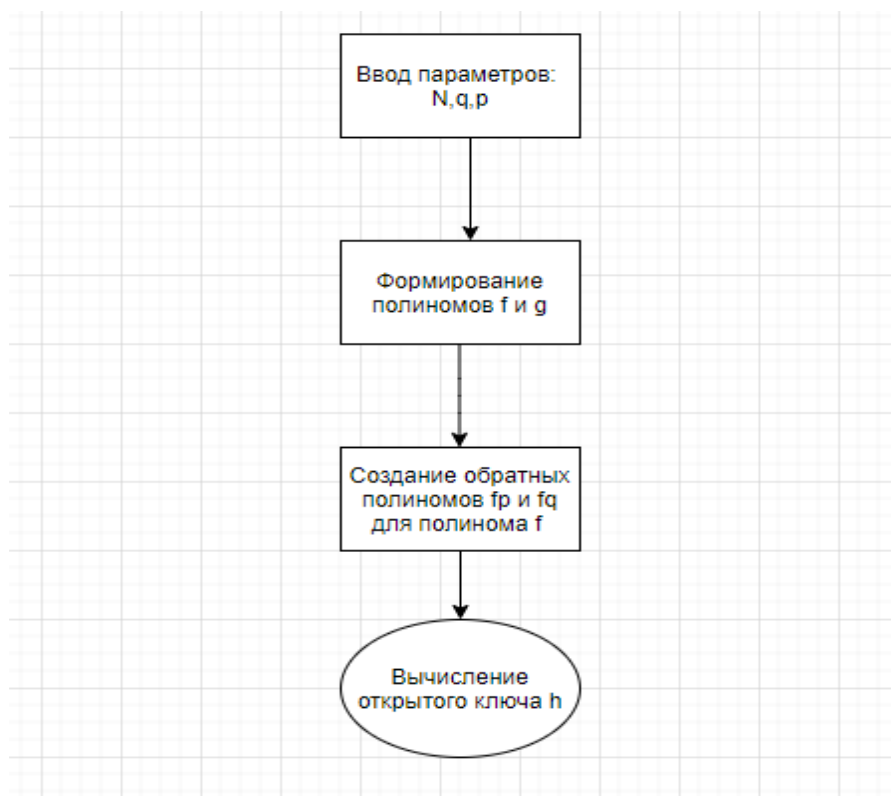


Рисунок 9 – Схема генерации открытого ключа

Далее пользователь выбирает сообщения, которое подлежит зашифровке с использованием открытого ключа. На рисунке 10 показана схема выполнения шифрования.

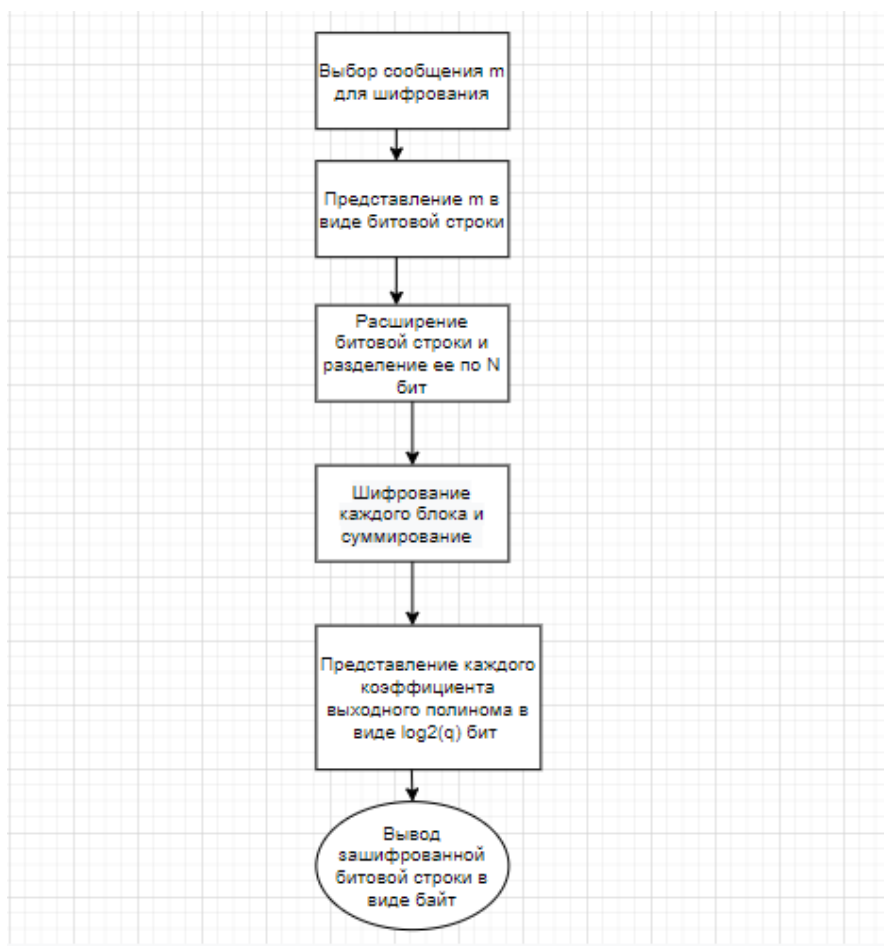


Рисунок 10 – Схема шифрования выбранного сообщения

После того как сообщения было зашифровано, данное программное средство позволяет его расшифровать, используя секретный ключ. При этом зашифрованное сообщение представляется в виде бит и далее битовая последовательность разбивается на блоки по N бит. После происходит расшифрования каждого блока и объединение всех расшифрованных блоков в общий массив.

Последний этап расшифрования зашифрованных данных является разбитие полученной битовой строки на символы и вывод на экран. На рисунке 11 изображена схема этого процесса.

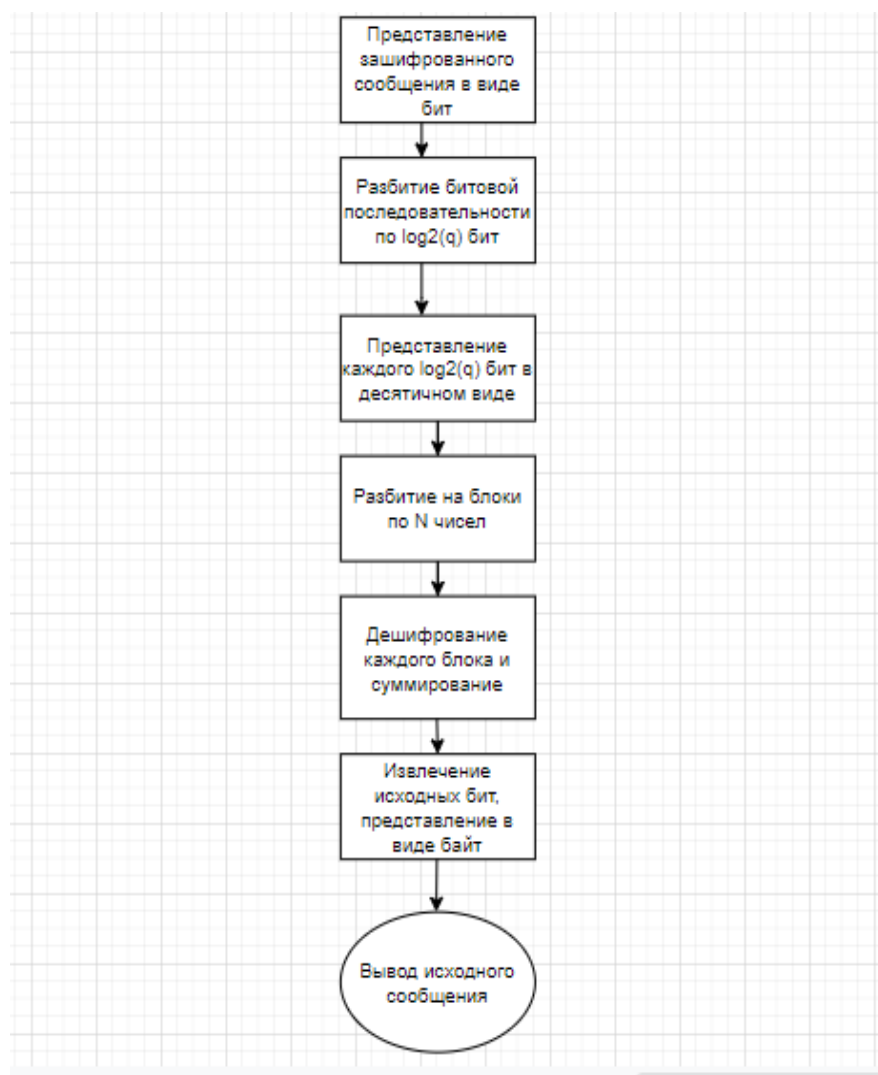


Рисунок 11 – Схема расшифрования выбранного сообщения

Вывод

Были описаны общие сведения криптографической системы NTRUEncrypt. Приведен принцип шифрования и расшифрования данной системы, а также формирования открытого и закрытого ключа. Дано описание выбранного языка программирования, структуры программы и приведена их обобщенная схема работы. Разработанные программные обеспечения представляют собой объектно-ориентированные приложения с простым и понятным интерфейсом. Приведены рисунки, демонстрирующие работу программного средства, в частности выбор параметров, выбор сообщения для шифрования и расшифрования зашифрованного текста.

4 Тестирование и демонстрация программного средства

4.1 Ввод параметров для генерации ключей

Общий вид диалогового окна программы показан на рисунках 12 и 13. В окне «Шифрование» пользователь задает параметры для генерации закрытого и открытого ключа в поле «Генерация ключей». После ввода параметров ключи сохраняются в папке «keys» и пользователь выбирает открытый ключ, а также задает «ослепляющий» полином для шифрования данных. В окне «Расшифрование» пользователь выбирает закрытый ключ, а также шифротекст для расшифрования. Закрытый и открытый ключ будут сохраняться в виде «key.priv» и «key.pub» соответственно.

The screenshot shows a window titled "NTRU" with a standard Windows title bar (minimize, maximize, close buttons). The window has two tabs: "Шифрование" (Encryption) and "Расшифрование" (Decryption). The "Шифрование" tab is active. Inside the window, there is a section titled "Генерация ключей" (Key Generation) with five input fields labeled "N", "p", "q", "df", and "dg". Below these fields is a button labeled "Сгенерировать" (Generate). To the right of this section is a large text area labeled "Зашифрованный текст:" (Encrypted text:). Below the key generation section, there are three input fields: "Открытый ключ" (Public key), "Полином g" (Polynomial g), and "Файл для шифрования" (File for encryption). Each of these fields has a "Выбрать" (Select) button next to it. Below these fields is a button labeled "Зашифровать" (Encrypt).

Рисунок 12 – Окно генерации ключей и шифрования

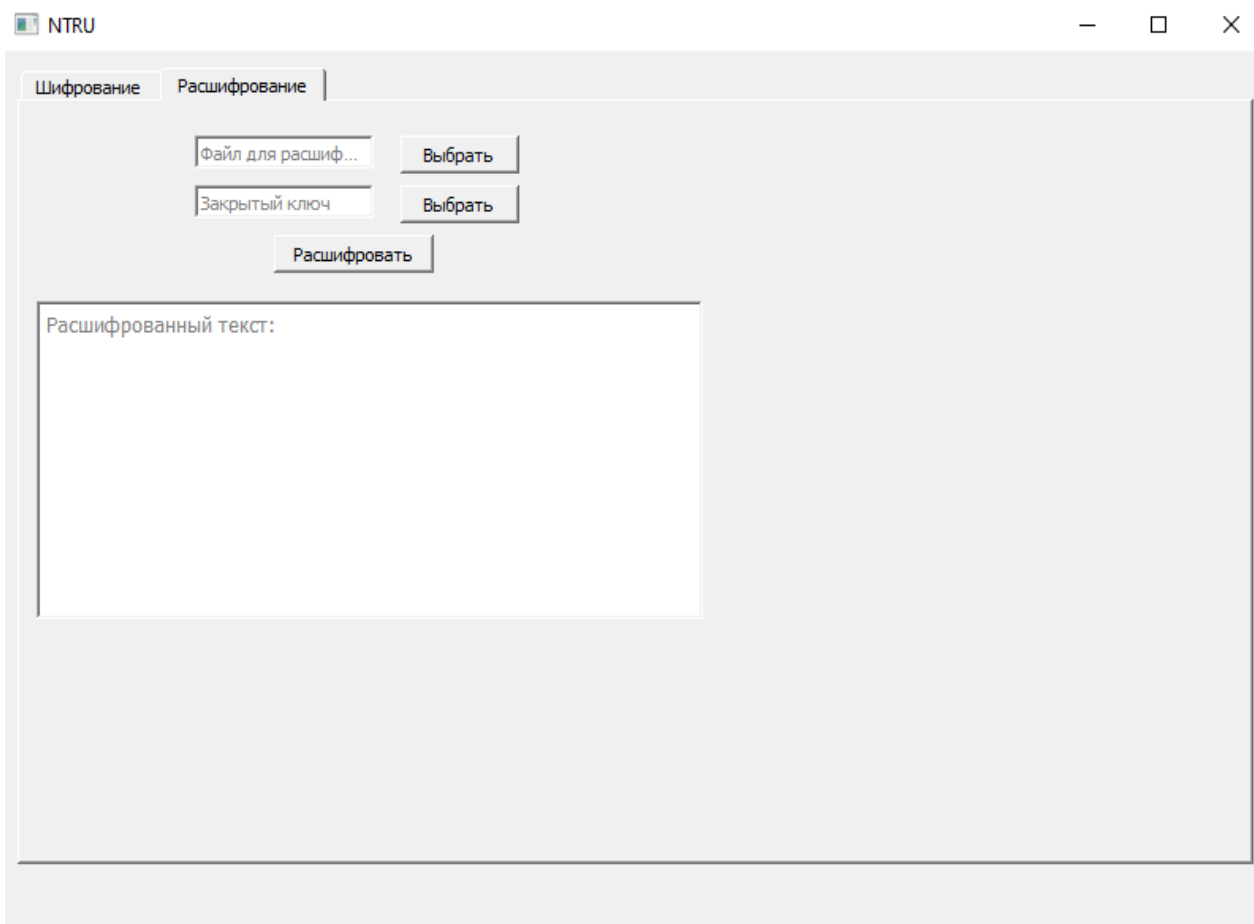


Рисунок 13 – Окно расшифрования

NTRUEncrypt использует шесть постоянных параметров: N , p , q , df , dg , dr . Эти параметры вводятся пользователем на панели «Шифрование», как показано на рисунке 14.

Число N характеризует размер открытого ключа, а также длину каждого блока данных. Целые числа p и q , у которых НОД (Наибольший общий делитель) (p, q) равен 1, не обязательно должны быть простыми. Параметр p предназначен для определения интервала, которому должны принадлежать все коэффициенты многочленов, используемых в криптографической системе. Параметры df и dg задают количество единиц в генерируемых программой полиномах f и g . Параметр dr задает число коэффициентов равных 1 и -1 «ослепляющего» полинома r , который используется при шифровании. Сначала пользователь вводит первые пять параметров, после чего происходит формирование полиномов f и g . После

этого происходит поиск обратных полиномов для f и g , однако может случиться так, что какой-то обратный полином не будет найден. Тогда программа сформирует другой полином и попытается найти обратный для него снова. Обратные полиномы вычисляются по формулам 6,7. Для достижения достаточного уровня криптостойкости все параметры должны соответствовать рекомендуемым.

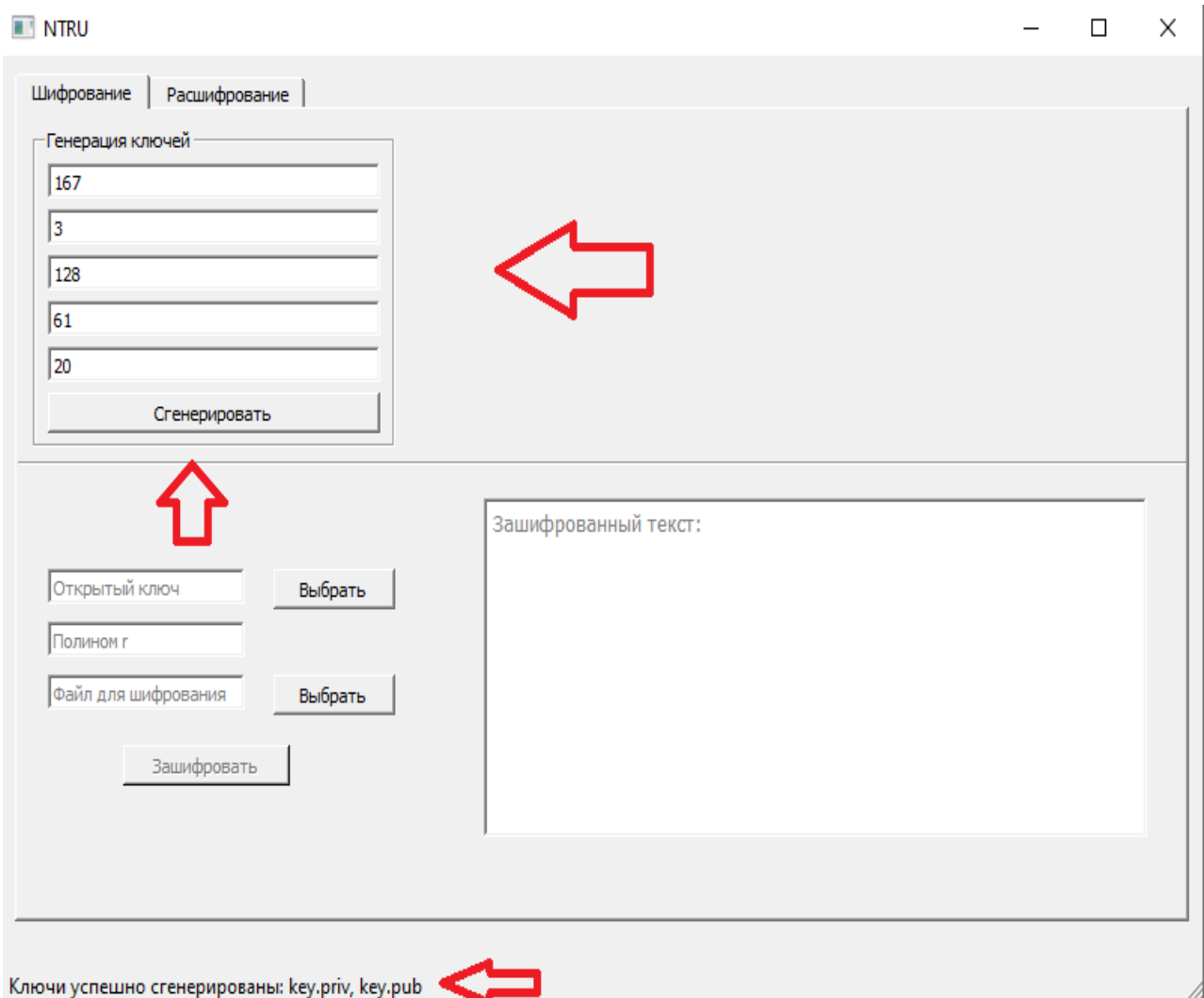


Рисунок 14 – Ввод параметров и генерация закрытого и открытого ключа

На рисунке 15 продемонстрированы сохраненные ключи шифрования в папке «keys». Открытый ключ «key.pub» понадобится для шифрования данных, а ключ «key.priv» для того, чтобы расшифровать полученный шифротекст.

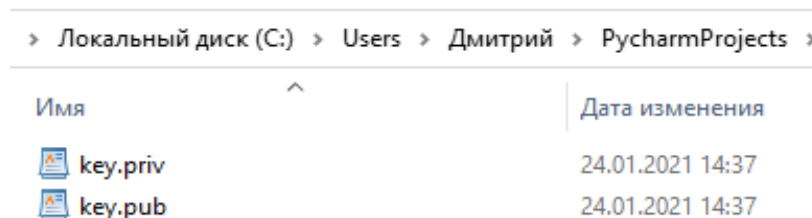


Рисунок 15 – Ключи шифрования

Также для удобства пользователь может видеть в окне вывода программы все полиномы, которые использовались при формировании ключей шифрования как показано на рисунке 16.

```
g: Poly(x**166 + x**163 + x**156 - x**153 + x**147 - x**146 + x**141 - x**137 - x**129 + 3
g coeffs: Counter({1: 20, -1: 20})
f: Poly(-x**166 + x**165 - x**164 + x**162 - x**161 - x**160 - x**158 - x**156 + x**155 -
f coeffs: Counter({1: 61, -1: 60})
f_p: Poly(-x**164 + x**163 - x**162 - x**161 + x**158 + x**156 + x**154 - x**153 - x**152
f_q: Poly(63*x**166 - 3*x**165 - x**164 + 40*x**163 - 62*x**162 - 58*x**161 - 33*x**160 +
h: Poly(58*x**166 + 39*x**165 + 46*x**164 - x**163 - 40*x**162 + 15*x**161 + 54*x**160 +
```

Рисунок 16 – Вывод использующихся полиномов при генерации ключей

4.2 Выбор файла и шифрование

Для того, чтоб пользователь мог зашифровать выбранный им файл, ему нужно указать число коэффициентов «ослепляющего» полинома g , открытый ключ, с помощью которого будет происходить шифрование и сам текстовый файл. Для шифрования информации используется формула 8.

Исходное сообщение содержится в файле text.txt и находится в том же файле, где и программное средство. Исходный текстовый файл имеет вид как показано на рисунке 17.

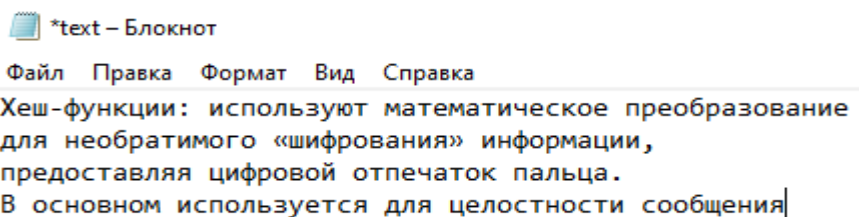


Рисунок 17 – Исходный текст

Стоит отметить, что исходное сообщение из файла разбивается на блоки по N бит, каждый из которых будет обрабатываться отдельно. После такого разбиения происходит генерация «ослепляющего» полинома g и шифрование каждого блока исходного сообщения. Для того, чтобы зашифровать данные в начале выполнится представление сообщения в виде битовой строки. Это необходимо сделать для того, чтобы можно было далее представить каждый блок в виде полинома, так как процесс шифрования представляет собой перемножение и сложение полиномов в кольце. Так как нельзя зашифровать полином степенью выше $N-1$, то полученная битовую строка расширяется таким образом, чтобы ее можно было разбить на блоки по N бит. Далее происходит шифрование каждого блока и получение коэффициентов, которые объединяются в единый массив. После этого происходит разбиение на байты и вывод полученного шифротекста. Выбор текстового документа, «ослепляющего» полинома и открытого ключа показан на рисунке 18.

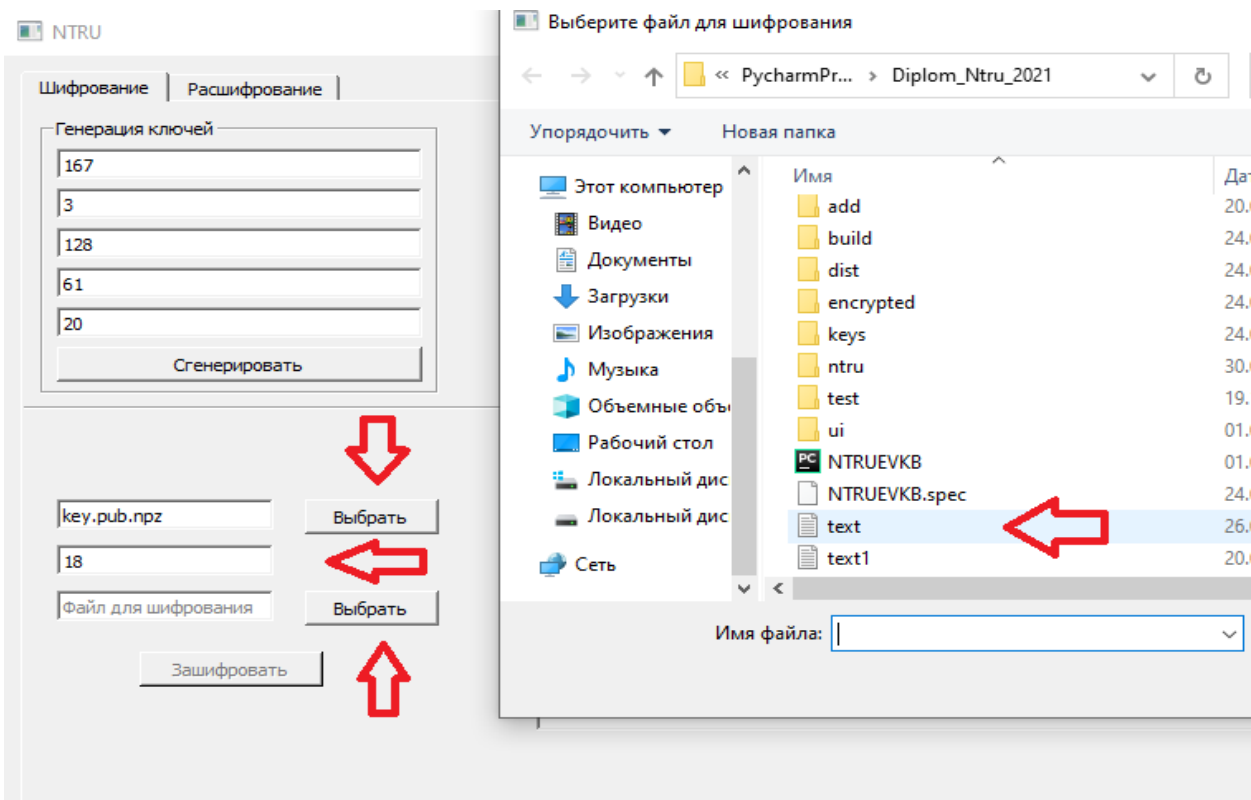


Рисунок 18– Выбор открытого ключа, полинома и текстового документа

Далее программа зашифрует файл, сохранит как «encrypted.enc» и выведет шифр после нажатия на кнопку «Зашифровать» как показано на рисунке 19.



Рисунок 19 – Вывод зашифрованного файла

Пользователь также может видеть какие полиномы использовались во время шифрования данных, как видно из рисунка 20.



Рисунок 20 – Вывод полиномов для шифрования

4.3 Расшифрование документа

Последним этапом выполнения программы является выбор зашифрованного документа и закрытого ключа как показано на рисунке 21. При расшифрования происходит снова представление зашифрованного текста в виде бит. Затем битовая последовательность представляется в виде коэффициентов, после чего происходит разбитие каждого ряда коэффициентов на блоки по N чисел. Каждый блок представляется в виде полинома и далее выполняется расшифрование с получением последовательности бит, которые разбиваются на байты. Таким образом полученный текст, совпадает с исходным. При расшифровании используются последовательно формулы 10-12.

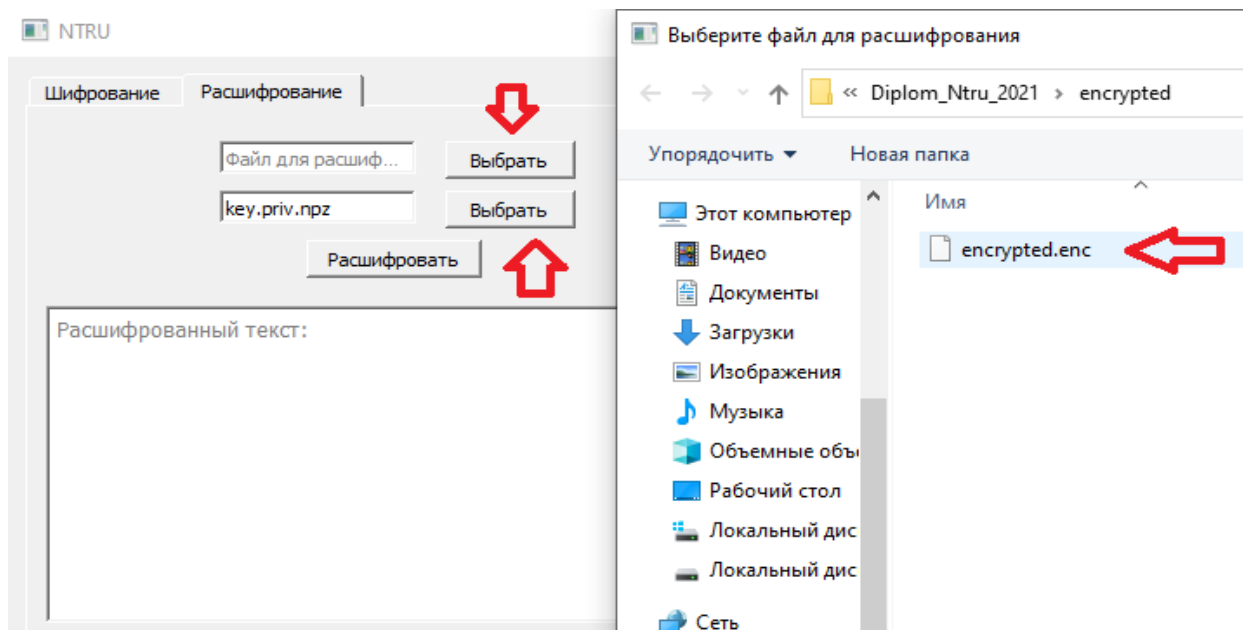


Рисунок 21 – Выбор зашифрованного текста и закрытого ключа

После нажатия на кнопку «Расшифровать» начнется расшифровка выбранного зашифрованного файла. Расшифрованный текст будет отображаться как показано на рисунке 22.

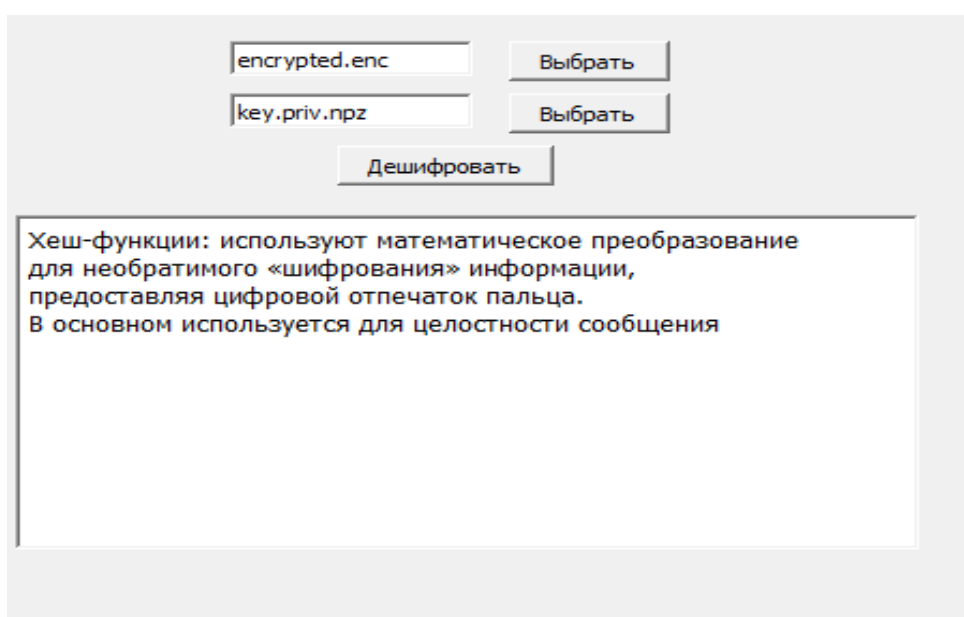


Рисунок 22 – Результат расшифрования

Кроме того, пользователь может видеть какие полиномы использовались при расшифровании в окне терминала, как показано на рисунке 23.

```

Encrypt message:Poly(126*x**166 + 21*x**165 + 4*x**164 + 10*x**163 + 72*x**162 + 9
a: Poly(-13*x**166 - 5*x**165 - 8*x**164 + 5*x**163 + 4*x**162 - 25*x**161 + 14*x
b: Poly(-x**166 + x**165 + x**164 - x**163 + x**162 - x**161 - x**160 - x**159 +
f: Poly(-x**165 + x**164 - x**162 + x**160 - x**158 - x**157 + x**155 - x**153 -
f_p: Poly(x**166 + x**165 + x**164 + x**163 - x**162 - x**161 - x**158 + x**157 -
Encrypt message:Poly(106*x**166 + 26*x**165 + 73*x**164 + 112*x**163 + 39*x**162 +
a: Poly(12*x**166 - 13*x**165 - 17*x**164 - x**163 + 15*x**162 + 9*x**161 + 8*x**
b: Poly(-x**165 + x**164 - x**163 - x**160 - x**158 + x**155 - x**154 - x**152 -
f: Poly(-x**165 + x**164 - x**162 + x**160 - x**158 - x**157 + x**155 - x**153 -
f_p: Poly(x**166 + x**165 + x**164 + x**163 - x**162 - x**161 - x**158 + x**157 -
Encrypt message:Poly(39*x**166 + 9*x**165 + 84*x**164 + 25*x**163 + 115*x**162 + 5
a: Poly(9*x**166 + 4*x**165 - x**164 - 21*x**163 - 4*x**162 - 5*x**161 - 6*x**160
b: Poly(x**165 - x**164 - x**162 + x**161 - x**159 - x**157 - x**156 - x**155 + x

```

Рисунок 23 – Вывод использующихся полиномов при расшифрования

4.4 Анализ разработанного программного продукта на зависимость длины ключа и скорости шифрования и расшифрования данных

На основе полученных в результате тестирования аналитических данных был выполнен сравнительный анализ скорости формирования ключа, шифрования и расшифрования данных реализованной криптографической системы NTRUEncrypt с другими криптографическими системами, а именно с RSA и ECC при различных параметрах, как показано на рисунке 24.

Криптографическая система	Время генерации ключа, мс	Время шифрования данных, мс	Время расшифрования данных, мс
NTRU 167	10.3	1.5	2.1
RSA 512	46.4	4.9	7.5
NTRU 263	19.8	3.4	4.5
RSA 1024	84.4	12.8	20.4
NTRU 503	51.6	6.8	9.6
RSA 2048	216.2	27.9	47.8
NTRU 251	17.7	2.6	3.4
ECC 192	60.3	10.1	6.9
NTRU 397	32.7	4.3	5.8
ECC 256	108.6	16.4	12,2
NTRU 263	19.8	3.4	4.5
ECC 168	54.4	13.1	9.9

Рисунок 24 – Сравнение скоростей генерации ключа, шифрования и расшифрования

Исходя из таблицы, разработанный программный продукт имеет преимущества в скорости как генерации ключа, так и в скорости шифрования и расшифрования данных по сравнению с другими криптографическими системами. Кроме того, разработанная система имеет более высокую криптостойкость за счет решеточной криптосистемы, на которой она основана и может стать достойной альтернативой доквантовых криптосистем, таких как RSA и ECC в постквантовую эпоху.

Вывод

Описан принцип формирования в программном средстве закрытого и открытого ключа шифрования, приведены формулы для генерации ключей, а также показан пользовательский интерфейс для ввода необходимых параметров. Также был описан принцип работы программного средства при шифровании текстового документа и приведен ряд формул, который использовался при этом. Кроме того, был описан алгоритм расшифрования зашифрованного текста и показан вывод программы после расшифровки выбранного документа и сравнена скорость генерации ключа, шифрования и расшифрования данных реализованной криптографической системы NTRUEncrypt с RSA и ECC.

Заключение

В данной работе была реализована система, предоставляющая ресурс шифрования и расшифрования данных, криптографическая стойкость которой объясняется сложностью решения задачи теории решеток, заложенной в основу алгоритма NTRUEncrypt. В рамках данной работы был описан алгоритм работы программного средства: выбор переменных для генерирования закрытого и открытого ключа шифрования, алгоритм шифрования и расшифрования информации с использованием сгенерированных ключей. Кроме этого, была построена схема работы программы с описанием каждого этапа выполнения реализуемого алгоритма и продемонстрирована работа программного средства с выводом зашифрованного и расшифрованного сообщения. Полная реализация криптографической системы NTRUEncrypt была выполнена на языке программирования Python. Важно отметить тот факт, что после создания квантового компьютера проблемы быстрого дискретного логарифмирования и факторизации будут в скором времени решены и такие алгоритмы как RSA, DSA и другие подобные станут не актуальными. Однако, криптографическая система NTRUEncrypt сохранит свою актуальность, так как не существует алгоритма, решающего проблему кратчайшего вектора решетки, это означает, что он вполне применим в «постквантовую» эпоху. В результате выполнения работы, были решены все поставленные задачи, а именно: проведен анализ предметной области, изучены уже существующие доквантовые криптографические системы, разработана программная реализация алгоритма NTRUEncrypt с интерфейсом пользователя, протестировано разработанное программное средство при различных параметрах и сравнена скорость генерации ключа, шифрования и расшифрования данных реализованного программного продукта с другими криптографическими системами. Таким образом цель работы была достигнута.

					<i>10.05.01.410000.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		53

Перечень использованных информационных ресурсов

1. Лапони́на, О. Р. Криптографические основы безопасности: учебное пособие / О. Р. Лапони́на – Москва: Национальный Открытый Университет ИНТУИТ, 2016. – 242 с. – Текст: непосредственный.

2. Ишмухаметов, Ш. Т. Методы факторизации натуральных чисел: учебное пособие / Ш. Т. Ишмухаметов. — Казань: Казанский Университет, 2011. – 190 с. – Текст: непосредственный.

3. Жданов, О. Н. Эллиптические кривые. Основы теории и криптографические приложения / О. Н. Жданов, В. А. Чалкин – СГАУ им. М. Ф. Решетникова. – Москва : URSS ЛИБРОКОМ, 2012 – 193 с. – Текст: непосредственный.

4. Болотов, А. А. Элементарное введение в эллиптическую криптографию. Алгебраические и алгоритмические основы / А. А. Болотов, С. Б. Гашков, А. Б. Фролов, А. А. Часовских – Москва : КомКнига, 2006 – 280 с. – Текст: непосредственный.

5. Жук, А.П. Защита информации: Учебное пособие / А.П. Жук, Е.П. Жук, О.М. Лепешкин, А.И. Тимошкин. – Москва: ИЦ РИОР, НИЦ ИНФРА - 2015. - 392 с. – Текст: непосредственный.

6. Методика определения угроз безопасности информации в информационных системах. – Текст: электронный // Методический документ ФСТЭК России: [сайт]. – 2015. – URL: [https://mindstep.ru/wiki/index.php/Методика определения угроз безопасности информации в информационных системах](https://mindstep.ru/wiki/index.php/Методика_определения_угроз_безопасности_информации_в_информационных_системах) (дата обращения 13.10.2020 г.).

7. Технологии защиты данных. – Текст: электронный // Search Info: [сайт]. – 2020. – URL: <https://searchinform.ru/informatsionnaya-bezopasnost/zaschita-informatsii/> (дата обращения 15.01.2021 г.).

8. Венбо, Мао. Современная криптография / М. Венбо: – Москва: Издательский дом «Вильямс», 2015. - 768 с. – Текст: непосредственный.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		54

9. Uttar Pradesh, Cryptanalysis of NTRU with n public keys / Allahabad, Uttar Pradesh [et al.]. – Text: electronic. // IEEE Access. – 2019. – P. 246-345. – DOI:10.1109/ISEASP.2017.7976980. – URL: <https://ieeexplore.ieee.org/document/79769> (date of the application 15.09.2020). – Access mode: free.

10. ShanYue Bu. Choosing Parameters for NTRU / ShanYue Bu, Hui Zhang [et al.]. – Text: electronic // IEEE Access. – 2020. – P. 154-230. – DOI: 10.1109/MINES.2009.133. – URL: <https://ieeexplore.ieee.org/document/5369016> (date of the application: 11.01.2021). – Access mode: free.

11. Weichi Yu, Study on NTRU Decryption Failures / Weichi Yu, Shixiong Zhu [et al.]. – Text: electronic. // IEEE Access. – 2019. – P. 116-305. – DOI: 10.1109/ICITA.2005.266. – URL: <https://ieeexplore.ieee.org/document/1489004> (date of the application 12.09.2020). – Access mode: free.

12. Bertels, K. Quantum Computer Architecture: Towards Full-Stack Quantum Accelerators. / K. Bertels, A. Sarkar, M. Serrao [et al.]. – Text: electronic. // IEEE Access. – 2018. – P. 102-492. – DOI: 10.23919/DATE48585.2020.9116502. – URL: <https://ieeexplore.ieee.org/document/9116502> (date of the application 11.08.2020). – Access mode: free.

13. NTRU CryptoLabs. NTRU Algorithms. – Text: electronic // NTRUEncrypt: [website]. – 2020. – URL: <http://ntru.com/cryptolab/> (date of the application 3.11.2020)

14. Cryptography Techniques. Basics of Cybersecurity. – Text: electronic // Educba: [website]. – 2020. – URL: <https://www.educba.com/> (date of the application 17.11.2020).

15. Quantum computing. Quantum supremacy. – Text: electronic // Wired: [website]. – 2019. – URL: <https://www.wired.co.uk/article/quantum-computing-explained> (date of the application 7.11.2020).

Приложение А

Техническое задание

СОГЛАСОВАНО

УТВЕРЖДЕНО

И.о. зав. каф. «КБИС»

(подпись) доц, к.т.н. Е.А. Ревякина

(подпись) к.т.н. Д.А. Короченцев

«___» _____ 2020 г.

«___» _____ 2020 г.

А.1 Введение

А.1.1 Наименование программы

Наименование программы – «Программная реализация криптографической системы NTRUEncrypt с открытым ключом».

А.1.2 Область применения

Областями применения данного программного средства могут быть сферы защиты информации.

А.2 Основания для разработки

Разработка ведется на основании приказа об утверждении тем дипломных работ, утвержденного Донским Государственным Техническим университетом.

А.3 Назначение разработки

А.3.1. Функциональное назначение

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		56

Функциональным назначением разработанного программного средства является полная реализация алгоритма NTRUEncrypt, позволяющая генерировать открытый и закрытый ключ, выполнять операции шифрования и расшифрования входных файлов данных.

А.3.2 Эксплуатационное назначение

Эксплуатационное назначение – использование программного обеспечения в организациях, для которых необходимо применение средств защиты данных для их безопасной передачи по сети.

А.4 Требования к программе

А.4.1 Требования к функциональным характеристикам

Выполняемая функциональность разработанного программного средства:

- выполнение операций шифрования и расшифрования сообщения;
- реализация функций генерации промежуточных полиномов – используются в модулях генерации ключей и шифрования исходного текста;
- реализация функций вычисления, открытого и секретного ключей;
- реализация функции шифрования исходного сообщения;
- реализация функции расшифрования шифротекста.

Входные данные для функционирования программного средства:

- параметры криптосистемы;
- путь к файлу, содержащему исходное сообщение.

А.4.2. Требования к надежности

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		57

Надежное функционирование программы должно быть обеспечено выполнением Заказчиком совокупности организационно-технических мероприятий, перечень которых приведен ниже:

- организацией бесперебойного питания технических средств;
- организацией правильного ввода параметров;
- регулярным выполнением требований ГОСТ 51188-98. Защита информации;
- испытания программных средств на наличие компьютерных вирусов.

А.4.3. Требования эксплуатации

Для функционирования программного продукта необходимо соблюдение всех требований и правил эксплуатации компьютерной техники. Дополнительных требований и ограничений не вводится. Требования к персоналу, работающему с данным программным продуктом: общие знания вычислительной техники и знание предметной области.

А.4.4 Требования к составу и параметрам технических средств

Для работы программы требуется наличие IBM PC – совместимого персонального компьютера с тактовой частотой процессора не менее 1,5 Гц, объемом свободной оперативной памяти не менее 256 Мб, монитором и клавиатурой. Необходимое место на жестком диске – 256 Мб.

А.4.5 Требования к информационной и программной совместимости

Программа должна работать под управлением операционной системы Windows 10 или более ранних версий. Базовый язык программирования – Python.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		58

А.4.6 Требования к маркировке и упаковке

Требований к маркировке и упаковке не предъявлялось.

А.4.7 Требования к транспортированию и хранению

Условия транспортирования, места хранения, условия складирования и сроки хранения в различных условиях должны соответствовать требованиям, предъявляемым к носителям информации на которых будет содержаться данное программное изделие.

Допустимы все способы транспортирования и хранения, не нарушающие целостность используемого носителя данных.

Программное средство может храниться на любом носителе информации, имеющее возможность подключения к ЭВМ.

А.5 Требования к программной документации

Программная документация должна включать следующие документы:

– документ «Техническое задание» (ГОСТ 19.201-78);

А.6 Стадии и этапы разработки

1. Системный анализ.
2. Общесистемное проектирование.
3. Подготовка технологических средств.
4. Программная реализация, рабочий проект.
5. Отладка программного средства в статике.
6. Тестовые испытания программного комплекса.

А.7 Порядок контроля и приемки

Контроль и приемка разработки осуществляется на основе испытаний контрольно-отладочных примеров. Примеры должны демонстрировать правильность работы используемых в программе структур данных и алгоритмов в различных ситуациях, которые могут возникнуть при выполнении программы. При этом проверяются выполнение всех функций программы и полнота документации.

Разработчик:

Жеделёв Дмитрий Валерьевич

Дата начала разработки:

« » 2020 г.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		60

Приложение Б

Руководство системного программиста

Б.1 Общие сведения о программе

Областью применения данного программного комплекса являются сферы разработки приложений, использующих передачу конфиденциальных данных по сети. Разработанный программный комплекс предоставляет пользователям возможность шифрования файла, содержащего конфиденциальные данные с возможностью последующего его расшифрования. На вход программе подается файл с данными с предварительным вводом параметров системы. Программное обеспечение должно быть размещено на рабочей станции в состав которой входят:

- центральный процессор Intel Pentium 1,5 ГГц;
- объем оперативной памяти – 512 Мб;
- свободное место на диске – 64 Мбайт;
- монитор или иное устройство отображения информации с аналогичными функциональными характеристиками;
- клавиатура или иное устройство ввода информации с аналогичными функциональными характеристиками;
- мышь или иной манипулятор с аналогичными функциональными характеристиками.

Б.2 Структура программы

Данное программное средство состоит из двух модулей:

- модуль формы, которая отвечает за взаимодействие пользователя с программой;

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		61

- модуль класса функций, реализующий операции над полиномами.

Б.3 Настройка программы

Программа предоставляет базовую функциональность сразу после своего запуска и не требует дополнительной настройки и конфигурирования. Кроме того, программа должны быть полностью укомплектована и должны быть соблюдены требования к аппаратной и программной среде, в которой выполняется программа.

Б.4 Проверка программного средства

Проверка работоспособности программы осуществляется при заданных входных параметрах для генерации ключа, параметров и текстового документа для шифрования, а также параметров и шифротекста для расшифрования.

Б.5 Сообщения системному программисту.

Сообщений системному программисту не предусмотрено, кроме необходимых комментариев в самом исходном коде программного комплекса.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		62

Приложение В

Руководство программиста

В.1 Общие сведения о программе

Областью применения данного программного комплекса являются сферы разработки приложений, использующих передачу конфиденциальных данных по сети. Разработанный программный комплекс предоставляет пользователям возможность шифрования файла, содержащего конфиденциальные данные с возможностью последующего его расшифрования. На вход программе подается файл с данными с предварительным вводом параметров системы.

Программное обеспечение должно быть размещено на рабочей станции в состав которой входят:

- центральный процессор Intel Pentium 1,5 ГГц;
- оперативное запоминающее устройства (ОЗУ), не менее 256 Мб;
- процессор с поддержкой x32 либо x64 набора инструкций;
- операционная система семейства Windows;
- платформа .NET Framework 2.0 – 4.5.2.
- монитор или иное устройство отображения информации с аналогичными функциональными характеристиками;
- клавиатура или иное устройство ввода информации с аналогичными функциональными характеристиками;

В.2 Характеристика программы

Программное средство реализует шифрование данных в формате .enc, генерацию ключей и расшифрование зашифрованных данных с использованием сгенерированного закрытого ключа. После запуска

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		63

программы необходимо установить параметры, сгенерировать ключи, а затем выбрать нужный файл.

В.3 Обращение к программе

Обращение к программе и работа с программой осуществляется путём запуска основного исполняемого модуля программы на машине под управлением одной из указанных в требованиях ОС, и, далее, осуществлением взаимодействия с программой через визуальный интерфейс.

В.4 Входные и выходные данные

В качестве входных данных работы программы являются параметры для генерации закрытого и открытого ключа, файл с расширением .txt, содержащий исходные данные, которые необходимо зашифровать. Выходными данными будут файлы: encrypted.enc – результат шифрования, а key.priv – закрытый ключ, key.pub – открытый ключ.

В.5 Сообщения

Сообщений системному программисту не предусмотрено, кроме необходимых комментариев в самом исходном коде программного комплекса.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		64

Приложение Г

Руководство оператора

Г.1 Общие сведения о программе

Областью применения данного программного комплекса являются сферы разработки приложений, использующих передачу конфиденциальных данных по сети. Разработанный программный комплекс предоставляет пользователям возможность генерации открытого и закрытого ключа, шифрования файла, содержащего конфиденциальные данные с возможностью последующего его расшифрования. На вход программе подается файл с данными с предварительным вводом параметров системы.

Г.2 Условия выполнения программы

Программное обеспечение должно быть размещено на рабочей станции в состав которой входят:

- оперативное запоминающее устройства (ОЗУ), не менее 256 Мб;
- процессор с поддержкой x32 либо x64 набора инструкций;
- операционная система семейства Windows;
- платформа .NET Framework 2.0 – 4.5.2.

Для успешного выполнения программы необходимо выполнение всех аппаратных и программных требований, а также необходимо использование корректных входных данных.

Г.3 Выполнение программы

Перед запуском программы необходимо подробно ознакомиться с алгоритмом, заложенным в основу программного обеспечения. После запуска

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		65

приложения оператору нужно указать параметры алгоритма для генерации открытого и закрытого ключа, файл для шифрования. Далее зашифровать выбранный текстовый документ и расшифровать, выбрав сгенерированный закрытый ключ, нажав последовательно на соответствующие кнопки пользовательского интерфейса. Все результаты работы выводятся на экран.

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		66

Приложение Д

Листинг

Листинг Д.1 – Генерация ключей

```
Class NTRUEApp(QtWidgets.QMainWindow,  
main_window.Ui_MainWindow):  
    def __init__(self):  
        super().__init__()  
        self.setupUi(self)  
        self.btnGenerate.clicked.connect(self.generate_keys)  
        # self.keys_count = int(len(os.listdir(KEYS_FOLDER))  
/ 2)  
        self.keys_count = 1  
        self.file_count = 1  
        self.file_to_encrypt: str = None  
        self.file_to_decrypt: str = None  
        self.public_key = None  
        self.private_key = None  
  
self.selectFileForEncryptBtn.clicked.connect(self.choose_file_to  
_encrypt)  
  
self.selectFileForDecryptBtn.clicked.connect(self.choose_file_to  
_decrypt)  
  
self.pubKeySelectBtn.clicked.connect(self.choose_public_key)  
  
self.selectPrivKeyBtn.clicked.connect(self.choose_private_key)  
        self.encryptBtn.clicked.connect(self.encrypt)  
        self.decryptBtn.clicked.connect(self.decrypt)  
  
self.rEdit.textChanged.connect(self.ready_to_encrypt)  
  
self.selectedPubKey.textChanged.connect(self.ready_to_encrypt)  
  
self.selectedFileForEncrypt.textChanged.connect(self.ready_to_en  
crypt)  
  
self.selectedPrivKey.textChanged.connect(self.ready_to_decrypt)  
  
self.selectedFileForDecrypt.textChanged.connect(self.ready_to_de  
crypt)  
  
    def keyR(self):  
        p = 12  
        g = gen_poly(self.N, self.dg) # генерация поля g с
```

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		67

```

коэффициентами dg
        print("g: {}".format(g))
        print("g coeffs: {}".format(Counter(g.coefs())))
        while p > 0 and (self.h_pubKey is None): # если
публич ключ создать вышло то выходим
            f_poly = gen_poly(self.N, self.df, k=-1) #
формирование поля f с коэф df
            # если инв не выйдет то возьмем другой f
            log.info("f: {}".format(f_poly))
            log.info("f coeffs:
{}".format(Counter(f_poly.coefs())))
            try:
                self.gen_pubKey(f_poly, g) # формирование
публичного ключа
            except NotInvertible:
                p -= 1
            if self.h_pubKey is None:
                raise Exception("невозможно сгенерировать
обратный полином")

    def generate_keys(self):
        N = int(self.NEdit.text())
        p = int(self.pEdit.text())
        q = int(self.qEdit.text())
        df = int(self.dfEdit.text())
        dg = int(self.dgEdit.text())
        NT = Class_new(df, dg, N, p, q) # в класс передаем
первые 5 пар
        try:
            NT.keyR() # функ ген ключей
            f = NT.f_poly.all_coefs()[::-1] # все коэф f
            polFp = NT.f_mod_p.all_coefs()[::-1] # все
коэф f(p)
            h = np.array(NT.h_pubKey.all_coefs()[::-1]) #
все коэф отк ключа h
            priv_key_name = f"key.priv"
            priv_key_path = os.path.join(KEYS_FOLDER,
priv_key_name)
            np.savez_compressed(priv_key_path, N=N, p=p,
q=q, f=f, f_p=polFp, df=df, dg=dg)
            pub_key_name = f"key.pub"
            pub_key_path = os.path.join(KEYS_FOLDER,
pub_key_name)
            np.savez_compressed(pub_key_path, N=N, p=p, q=q,
h=h)

            self.statusbar.showMessage(f"Ключи успешно
сгенерированы: {priv_key_name}, {pub_key_name}", 10000)
            print("Ключи успешно сгенерированы")
        except Exception as e:
            print("Ошибка при генерации ключей")
            self.statusbar.showMessage(f"Ошибка при

```

генерации ключей: {e}", 2000)

Листинг Д.2 – Шифрование данных

```
def encrypt(self):
    with open(self.file_to_encrypt, "rb") as f:
        data = f.read()
        mess_arr = np.unpackbits(np.frombuffer(data,
dtype=np.uint8)) # представляем текст
        # в виде битовой строки
        # mess_arr = Numpy.trim_zeros(mess_arr, 'b')
        r = int(self.rEdit.text())
        output = ntru.cipher.encrypt(r, 0, 0, self.public_key,
mess_arr)
        res =
np.packbits(np.array(output).astype(np.int)).tobytes() #
представление в виде байт
        filename = os.path.basename(self.file_to_encrypt)
        with open(os.path.join(ENCRYPTED_FILES_FOLDER,
f"encrypted.enc"), "wb") as f:
            f.write(res)

self.ecnryptedTextBrowser.setText(base64.encodebytes(res).decode
())

self.keys_count += 1

def encrypt(r, df, dg, pub_key, arr): # шифрование
    # keyPub = Numpy.load(pub_key, allow_pickle=True) #
подгружение файла pub_key в котором
    # хранится N p q h
    Nt = Class_new(df, dg, int(pub_key['N']), int(pub_key['p']),
int(pub_key['q']))
    Nt.h_pubKey = Poly(pub_key['h'].astype(Numpy.int)[::-1],
x).set_domain(
ZZ) # тк keyPub['h'] хранится как массив коэф
    # то его нужно снова представить как полином
    arr = Add(arr, Nt.N) # расширение битовой строки
    arr = arr.reshape((-1, Nt.N)) # разбиение на блоки по N
частей
    res_mass = Numpy.array([]) # создание пустого списка
    for _, curr_part in enumerate(arr, start=1):
        curr_msg = Poly(curr_part[::-1], x).set_domain(ZZ) # N
представить первые N бит в виде полин и переве
        curr_poly_r = gen_poly(Nt.N, r) # генерация полинома r
        curr_mass = Nt.encrypt_part(curr_msg,
curr_poly_r).all_coeffs()[::-1] # зашифрованный блок
        if len(curr_mass) < Nt.N:
            curr_mass = Numpy.pad(curr_mass, (0, Nt.N -
len(curr_mass)), 'constant')
```



```

        res_mass = Numpy.concatenate((res_mass, curr_mass)) #
объединение блоков в один
        # разбиение каждого коэффициента по log2(q) бит и
объединение
        res_mass = [[0 if s == '0' else 1 for s in
Numpy.binary_repr(n, width=int(math.log2(Nt.q)))] for n in
res_mass]
        return Numpy.array(res_mass).flatten()

```

Листинг Д.3 – Расшифрование данных

```

def decrypt(self):
    with open(self.file_to_decrypt, "rb") as f:
        data = f.read()
        byte_to_bin = np.unpackbits(np.frombuffer(data,
dtype=np.uint8)) # представлние посл байт в биты
        byte_to_bin = np.trim_zeros(byte_to_bin, 'b') #
удаление нулей в конце, часть которых нужно будет вернуть
        res = ntru.cipher.dec(self.private_key, byte_to_bin)
        raw_bytes =
np.packbits(np.array(res).flatten().astype(np.int)).tobytes()
        text = raw_bytes.decode('utf-8', errors='ignore')
        self.decryptResult.setText(text)

def dec(private_key, arr_dec):
    # pr_new_key = Numpy.load(prKey, allow_pickle=True)
    Nt = Class_new(0, 0, int(private_key['N']),
int(private_key['p']), int(private_key['q']))
    Nt.f_poly = Poly(private_key['f'].astype(Numpy.int)[::-
1], x).set_domain(ZZ)
    Nt.f_mod_p =
Poly(private_key['f_p'].astype(Numpy.int)[::-1],
x).set_domain(ZZ)
    m = int(math.log2(Nt.q))
    m1 = m - len(arr_dec) % m # вычисляем число нулей
которые нужно вернуть
    if m1 == m:
        m1 = 0

    # возвращаем недостающие нули и делим битовую строку по
log2(q) бит
    # после чего получаем коэффициенты
    arr_dec = Numpy.array(
        [int("".join(n.astype(str)), 2) for n in #
"".join(n.astype(str)), 2) соберет по log2(q) бит
        # и превратит в коэф-т
        Numpy.pad(Numpy.array(arr_dec), (0, m1),
'constant').reshape((-1, int(math.log2(Nt.q))))])
    # reshape разобьет на группы по log2(q) бит

```

```

arr_dec = arr_dec.reshape((-1, Nt.N)) # разбиение
послед-и коэф на блоки по N частей
res_mass = Numpy.array([])
for _, curr_part in enumerate(arr_dec, start=1):
    curr_mass_dec = Nt.dec_block(Poly(curr_part[::-1],
x).set_domain(ZZ)).all_coeffs()[::-1]
    if len(curr_mass_dec) < Nt.N:
        curr_mass_dec = Numpy.pad(curr_mass_dec, (0,
Nt.N - len(curr_mass_dec)), 'constant')
    res_mass = Numpy.concatenate((res_mass,
curr_mass_dec))
return add_dec(res_mass, Nt.N)

```

					10.05.01.410000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		71